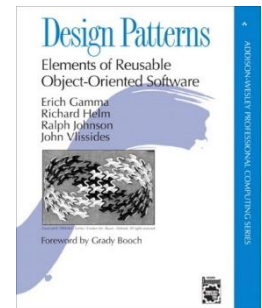


SOFTWARE DESIGN PATTERNS

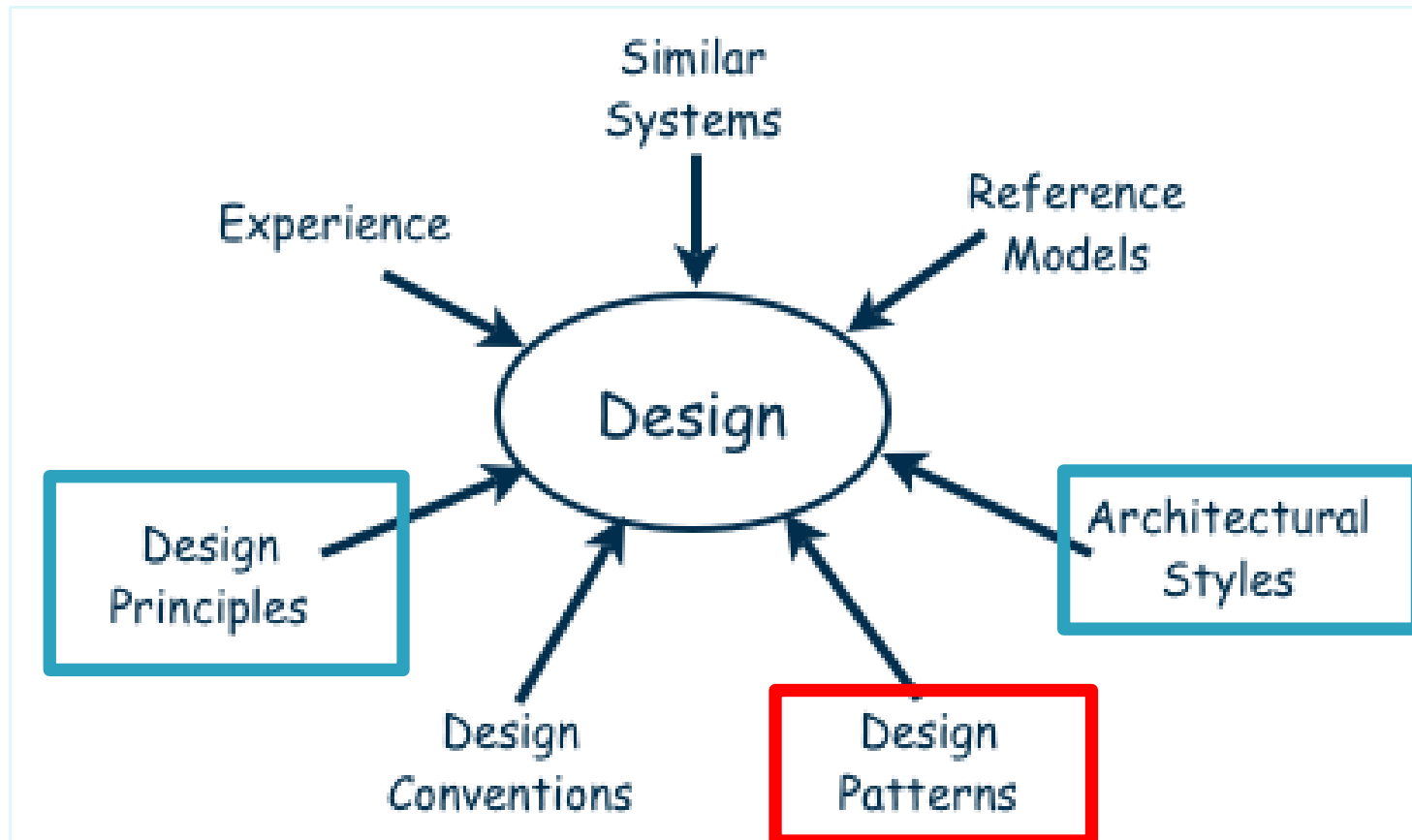
Ingegneria del Software a.a. 2025-26

AGENDA

- Cos'è un **Design Pattern**?
 - Differenza con Framework e Libreria
- Pattern elementari **GRASP**
 - General Responsibility Assignment Software Patterns
- Elementi definiti nei design pattern:
 - Nome, Problema, Soluzione, Conseguenze
- **Catalogo** di design patterns **GoF**:
 - Creational Patterns, Structural Patterns, Behavioral Patterns
- GoF patterns che vedremo nel corso (**6/23**):
 - Abstract Factory, Adapter, Facade, Template Method, Observer, State

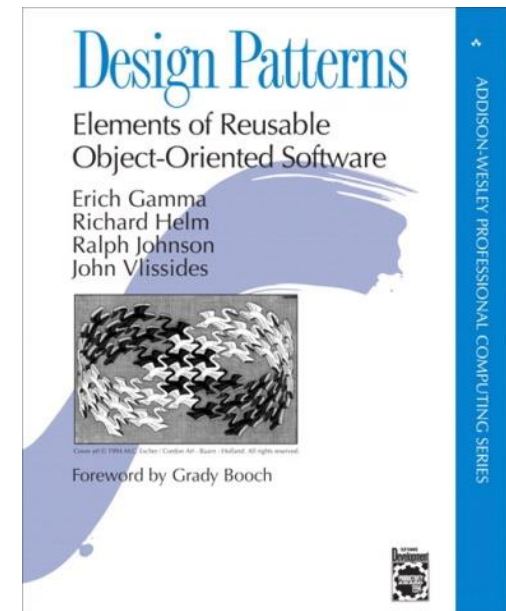


DESIGN: UN PROCESSO CREATIVO?



DESIGN PATTERN

- Una **soluzione elegante** (con un **nome**) di uno **specifico problema** di design/programmazione OO che costituisce un **unità di riuso**
 - Soluzioni ampiamente “levigate” ed usate (esperienza)
 - Non le soluzioni che vengono in mente per prime
 - Non basate su un particolare linguaggio OO
- L'idea di design pattern deriva da **Kent Beck** (famoso anche per Extreme Programming)
- Formalizzati nel libro autorevole e di larga diffusione GoF (Gang Of Four) 1995
- Attualmente molti cataloghi di pattern, in genere relativi ad:
 - un particolare dominio applicativo
 - Es. Game programming patterns
 - una particolare tipologia di applicazioni
 - Es. Web applications, Mobile and Enterprise



DESIGN PATTERN: A COSA SERVONO?

- Aiutano ad applicare i principi di **buona** progettazione OO
 - Es. high cohesion - low coupling
- Aiutano a creare “buoni” design
 - Possono essere applicati sia durante la **modellazione** che la **codifica**
- Facilitano la comunicazione tra sviluppatori

Jack il sottosistema della persistenza deve esporre i suoi servizi come una *Facade*. Useremo un *Abstract Factory* per i *Data Mapper*, e i *Proxy* per la Lazy initialization ...



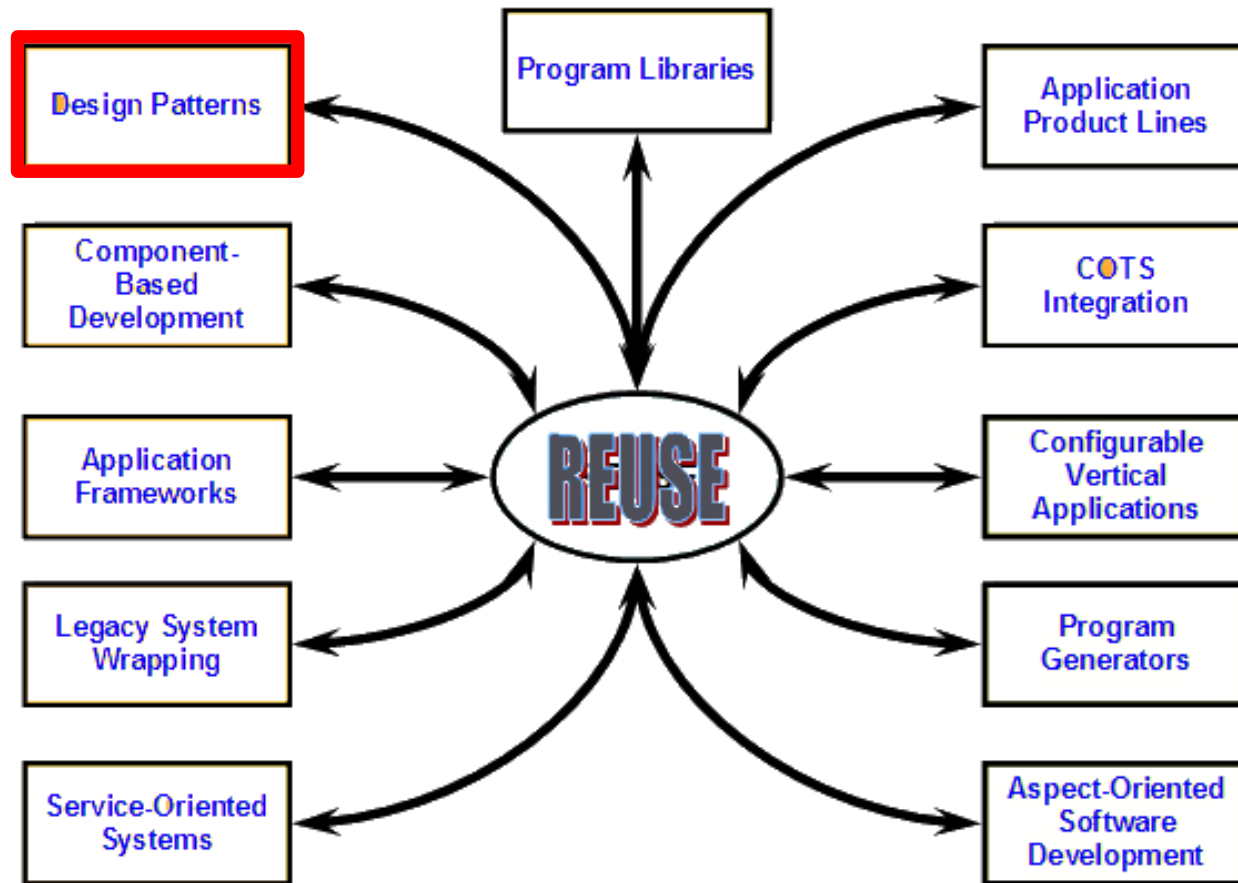
Ehh? Cosa accidenti hai detto?



ALTRE UNITÀ DI RIUSO: IL PANORAMA

Non l'unica!

[da Sommerville]



Perché il riuso?

È una best practice ...

ALTRE UNITÀ DI RIUSO

○ Librerie

○ Componenti riusabili (COTS)

- Parti di applicazione riusabili con interfaccia ed implementazione (non visibile)

○ Framework

COTS = (Commercial) Off-the-Shelf component

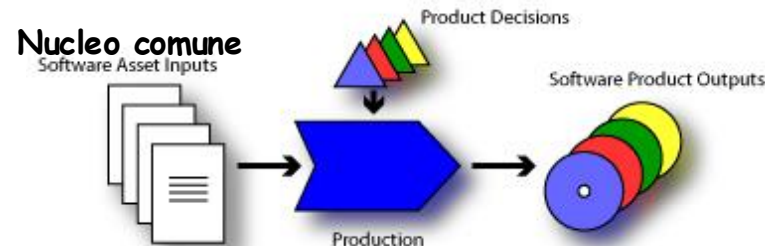
- Applicazione “con buchi” (assumono il controllo dell’applicazione)
 - Spesso per capire come funziona un framework bisogna conoscere i Design pattern ...

○ Product line

- **Famiglie di prodotti** sviluppate fin dall’inizio in contemporanea, con nucleo comune (core asset base)
 - Es. Visual Paradigm

○ Modelli

- es. del dominio, dei requisiti, del design



“Reuse the main body and write code it calls”

FRAMEWORK

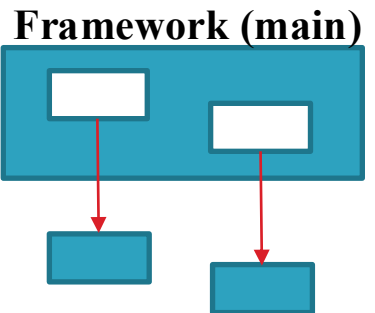
- Un framework è un insieme di classi e interfacce cooperanti che realizzano un **design riusabile** e **customizzabile** per uno specifico dominio applicativo o tipologia di app (GUI-based app, web applications, accounting systems, ...)

- Dettano l'architettura:
 - Inversione del flusso di controllo
- Customizzati creando sottoclassi specifiche dell'applicazione

- **Esempi di framework nel mondo Java:**

- Struts / Struts 2 – per Web app
- Spring – per JAVA app (anche Web)
- Hibernate – per gestire oggetti persistenti

Struts²



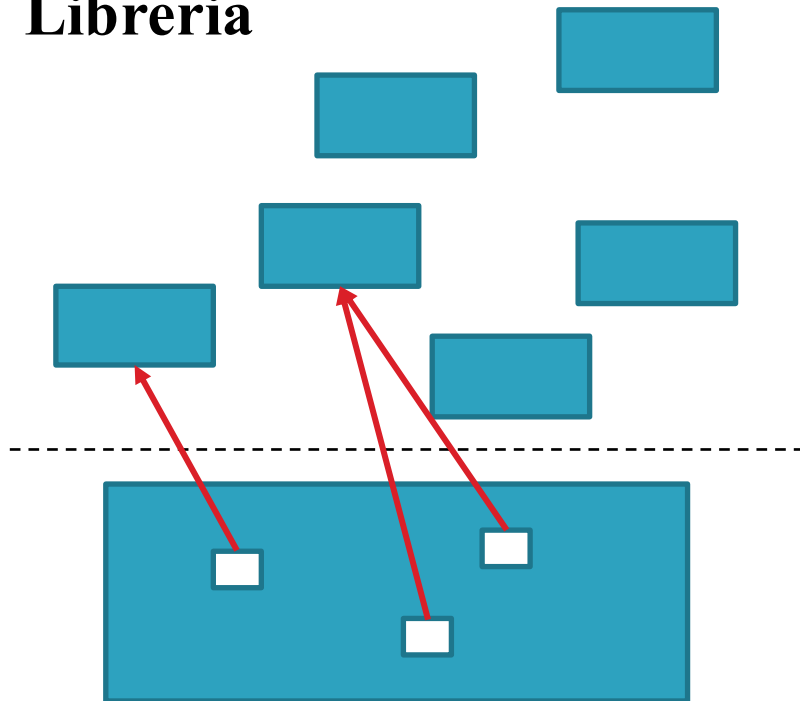
Codice da scrivere

spring

HIBERNATE

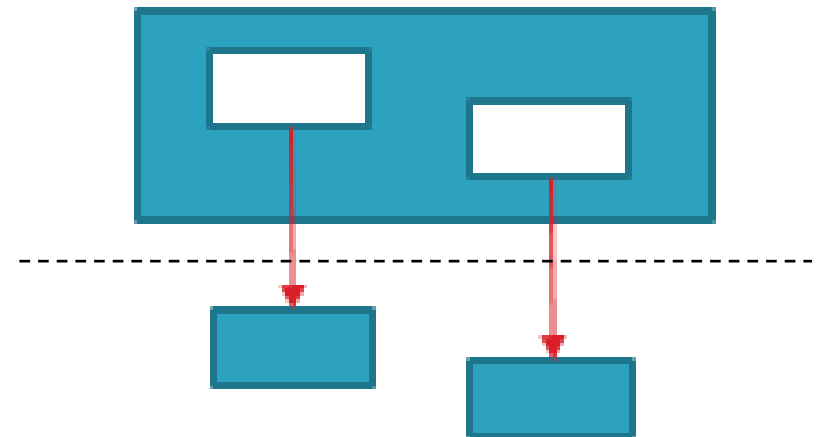
LIBRERIA VS. FRAMEWORK

Libreria



Codice da scrivere (main)

Framework (main)



Codice da scrivere

FRAMEWORK VS. 'XYZ'

◦ Framework vs. libreria software:

- **libreria:** lo sviluppatore di un'applicazione chiama le operazioni delle classi di un libreria
- **framework:** lo sviluppatore di un'applicazione scrive le classi (in particolare le operazioni) che vengono chiamate (usate) dal framework
 - “lo sviluppatore riempie i buchi”

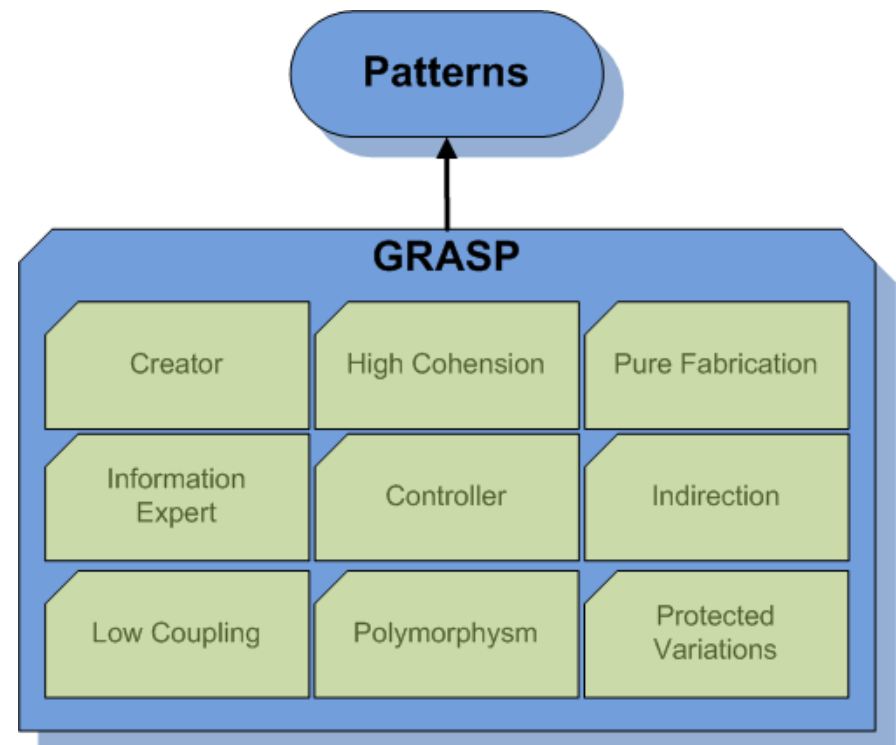
◦ Design pattern vs. framework

- più astratti
 - vanno implementati/adattati non si possono usare “as-is”
- più piccoli (come elementi architettureali)
- di solito meno specializzati, **non relativi a dominio applicativo specifico o tipologia di app**
 - Es. Design pattern ‘Controller’ si può usare sia in mobile che Web app

DESIGN PATTERN ELEMENTARI: GRASP

- I pattern Grasp sono pattern per **l'assegnazione di responsabilità** nel software
- Costituiscono **il fondamento** per la progettazione di sistemi OO

*Applicare UML e i pattern
a cura di Luca Cabibbo
Pearson; 3° edizione*



DESIGN PATTERN CLASSICI

Quelli del “libro” originale GoF

- 23 Design Pattern



Tre categorie:

- **Creazionali**

- creazione degli oggetti (in modo controllato)

- **Strutturali**

- composizione di classi ed oggetti (strutture tipiche)

- **Comportamentali**

- come classi (oggetti) interagiscono tra di loro e si distribuiscono le responsabilità

SPAZIO DEI DESIGN PATTERN

<i>Creazione</i>	<i>Struttura</i>	<i>Comportamento</i>
Factory Method Abstract Factory Builder Prototype Singleton	Adapter Bridge Composite Decorator Facade Flyweight Proxy	Interpreter Template Method Chain of responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

DESCRIZIONE DI UN PATTERN

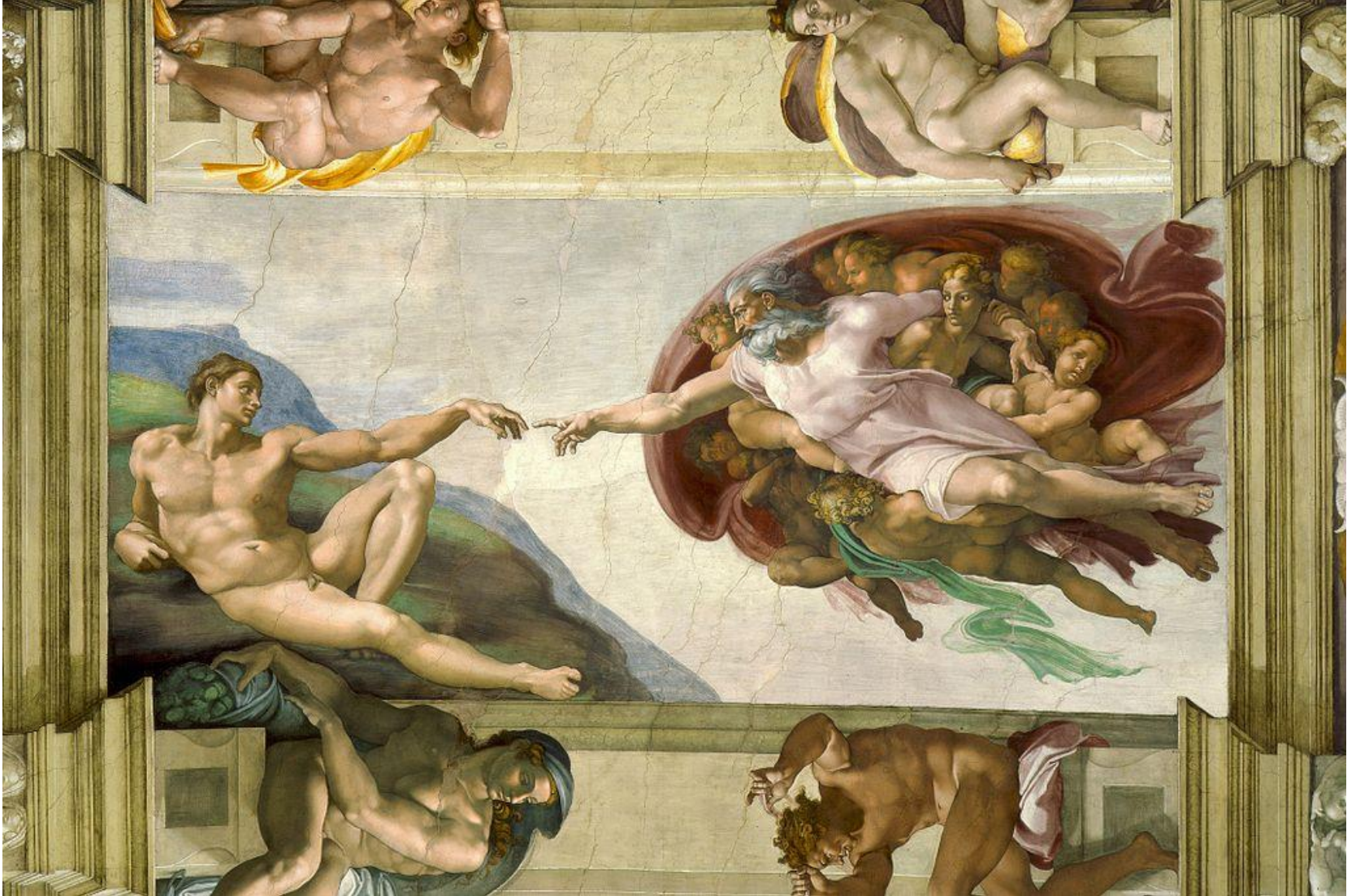
- Template piuttosto preciso!
 - Versione semplificata (solo parti fondamentali)
- **Nome (una/due parole)**
 - Significativo/esplicativo, che aiuta a capire a cosa serve il pattern
- **Problema**
 - describe **quando** applicare il pattern
 - può contenere anche delle precondizioni per poterlo applicare
- **Soluzione** **Non sempre precisa e completa, è uno sketch**
 - describe **astrattamente** gli elementi del design proposto, le loro relazioni, responsabilità e collaborazioni
- **Conseguenze**
 - Pro/contro della sua applicazione



INIZIAMO CON ...



INIZIAMO CON ...



I PATTERN CREAZIONALI



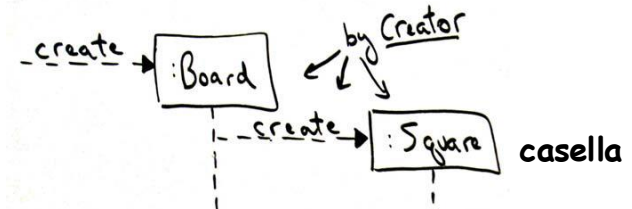
- Sono pattern che hanno a che fare con la creazione di istanze

Chi crea un oggetto di tipo A?

- Di solito per risolvere questo problema ci si affida al pattern GRASP **Creator**

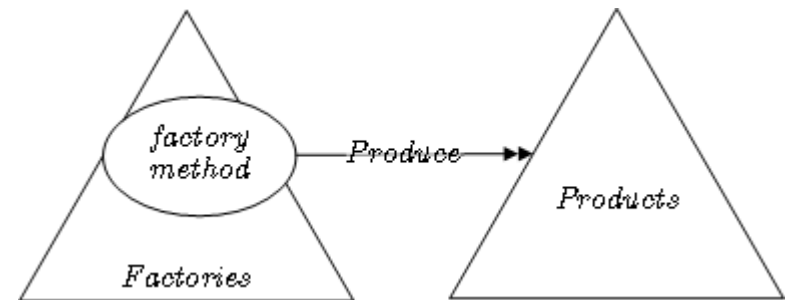
- Assegna alla classe B la responsabilità di creare un istanza della classe A se una delle condizioni è vera (più sono vere meglio è)

- B contiene o aggrega oggetti di tipo A
- B utilizza strettamente A
- B possiede i dati per l'inizializzazione di A



- Tuttavia esistono casi in cui non conviene applicare il pattern **Creator** ...

FACTORY



- Quando si vuole **nascondere la logica di creazione** perchè potenzialmente complessa
- Quando si vuole separare la logica di creazione dalla logica applicativa pura
- Perchè si vogliono introdurre strategie per la gestione della memoria per migliorare le prestazioni
 - Caching o 'riciclaggio' di oggetti
- In questi casi si ricorre al concetto di **Factory** (fabbrica), in cui **viene definito un oggetto Factory per creare gli oggetti ...**



ABSTRACT FACTORY



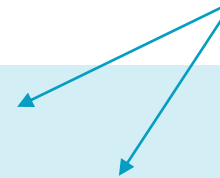
- Fornisce una soluzione **per creare famiglie di prodotti** (oggetti connessi o dipendenti tra loro), in modo che non ci sia necessità da parte dei **client** di specificare le classi concrete dei prodotti all'interno del proprio codice

Famiglia prodotti “1” (widget): **Window1**, **ScrollBar1**, ...

Client (GUI)

```
Window w = new Window1();  
...  
ScrollBar s = new ScrollBar1();
```

NO!!!



QUANDO USARE ABSTRACT FACTORY?

- Un sistema deve essere indipendente da come i suoi prodotti **sono creati**, composti e rappresentati
- Un sistema deve essere **configurato** per **una di diverse** famiglie di prodotti disponibili
 - Ad esempio deve poter funzionare con:
 - Famiglia 1: Window1, ScrollBar1, ...
 - Famiglia 2: Window2, ScrollBar2, ...
- Un famiglia di prodotti collegati tra loro è progettata per funzionare assieme, e bisogna rispettare questo vincolo
 - Es. Window1 e ScrollBar1 (e non Window1 e ScrollBar2)

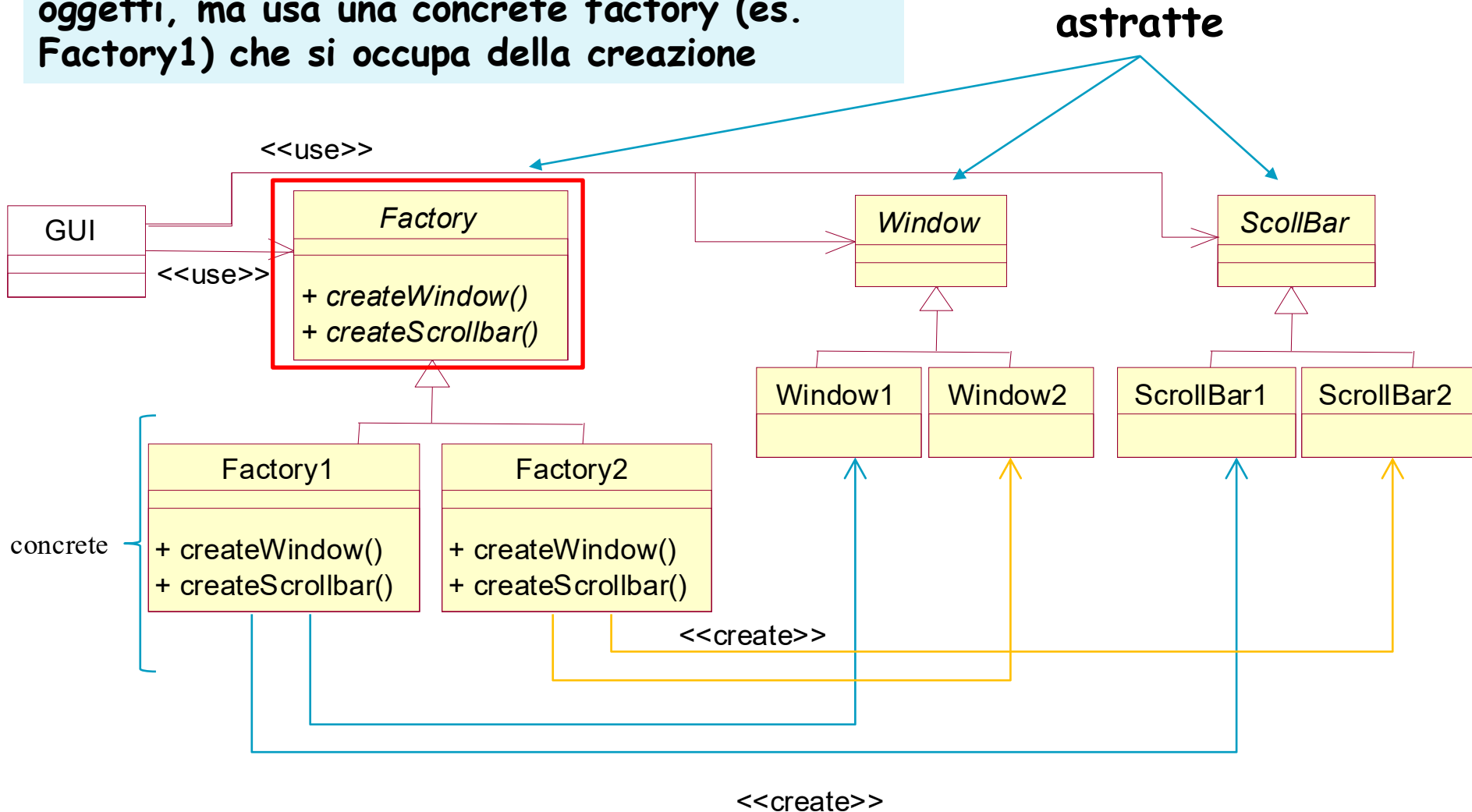
ESEMPIO/PROBLEMA

- Vogliamo progettare una GUI
 - **Multipiattaforma**
 - Se eseguita su piattaforma 1, istanzia prodotti della famiglia “1”: **Window1**, **ScrollBar1**, ...
 - Se eseguita su piattaforma 2, istanzia prodotti della famiglia “2”: **Window2**, **ScrollBar2**, ...
 - **Però che non dipenda troppo dalla piattaforma**
 - La GUI (che creerà/istanzierà quelle classi) non deve sapere quali classi concrete istanzia
 - Bisogna evitare che la GUI accoppi (sbagliando) per esempio **Window1** e **ScrollBar2**

Usiamo una “fabbrica astratta”!

LA FABBRICA DEGLI OGGETTI

La GUI non chiama i costruttori per i vari oggetti, ma usa una concrete factory (es. Factory1) che si occupa della creazione



COSA FA LA GUI?

- Senza la fabbrica:

```
Window w = new Window1();  
...  
ScrollBar s = new ScrollBar1();
```

- Con la fabbrica:

```
Factory f = new Factory1();  
Window w = f.createWindow();  
...  
ScrollBar s = f.createScrollBar();
```

DIFFERENZE

Senza la “fabbrica”

- GUI deve “conoscere” **Window1** e **ScrollBar1**
- GUI crea una **Window1**
- Poi crea una **ScrollBar1**
- GUI ha la responsabilità di accoppiare correttamente **Window1** e **ScrollBar1**

Con la “fabbrica”

- GUI chiede una fabbrica della famiglia “1”
- Poi chiede alla fabbrica una **Window**
- Poi chiede alla fabbrica una **ScrollBar**
- La responsabilità di accoppiare **Window1** e **ScrollBar1** è delegata alla fabbrica

○ Separazione delle responsabilità

- Disaccoppia la responsabilità della creazione (nella fabbrica)...
- ... dalla responsabilità dell'uso dei prodotti (nel cliente)

○ Perché “astratta”?

- La GUI conosce/usa solo classi astratte dei prodotti

DIAGRAMMA ABSTRACT FACTORY

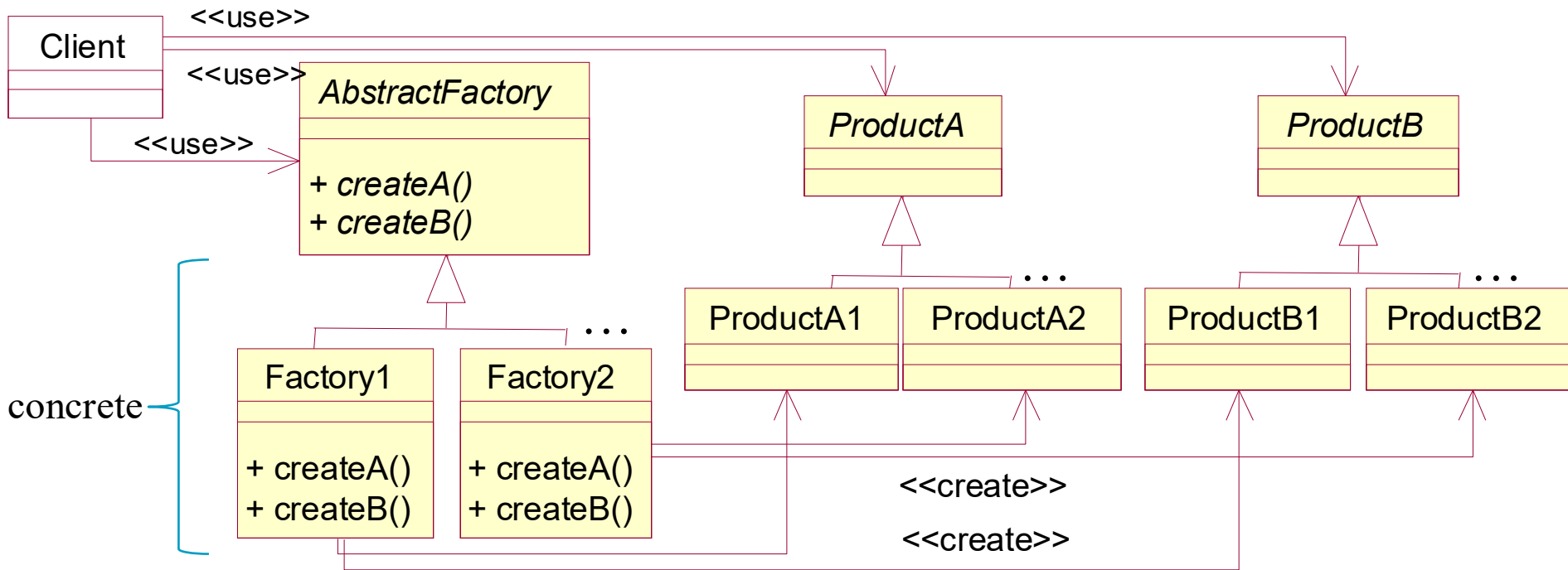


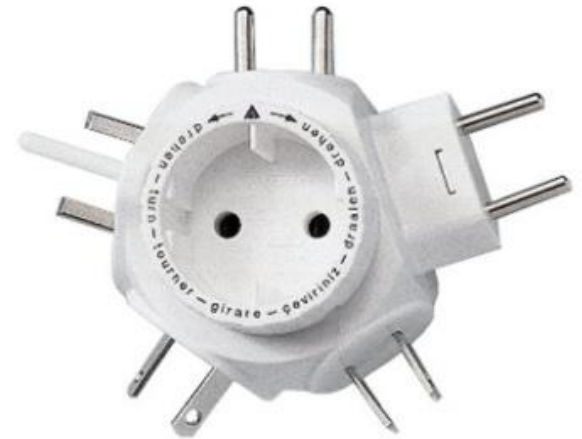
Diagramma che si trova nei cataloghi

ABSTRACT FACTORY

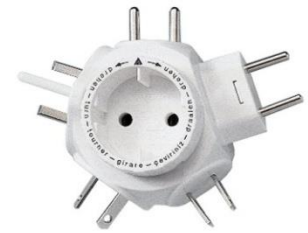
○ Conseguenze:

- Isola le classi concrete 😊
 - Il client (la GUI nel nostro esempio) non deve sapere niente dei prodotti che userà, neanche al momento dell'istanziamento dei prodotti
- Rende il cambio della famiglia dei prodotti facile 😊
 - basta cambiare la sola classe ConcreteFactory (una linea di codice)
- Favorisce la consistenza tra i vari prodotti di una famiglia (es. `Window1` e `ScrollBar1`) 😊
- Non è facile aggiungere nuovi prodotti 😞
 - Può richiedere cambiamenti all'interfaccia dell'Abstract Factory e alle sue sottoclassi
 - Richiede l'aggiunta di una nuova classe ConcreteFactory e di nuovi AbstractProducts e Products.

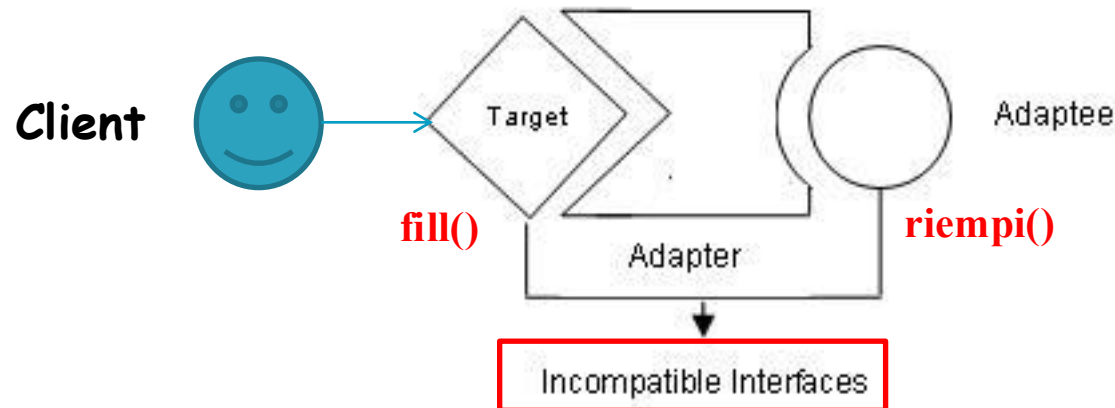
ADAPTER



ADAPTER



- **Converte l'interfaccia di una classe in un'altra interfaccia che il cliente si aspetta**
- Permette a delle classi di lavorare assieme, anche se non potrebbero visto che hanno interfacce incompatibili ...

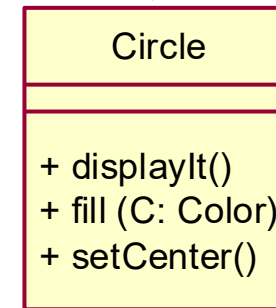


○ Problema

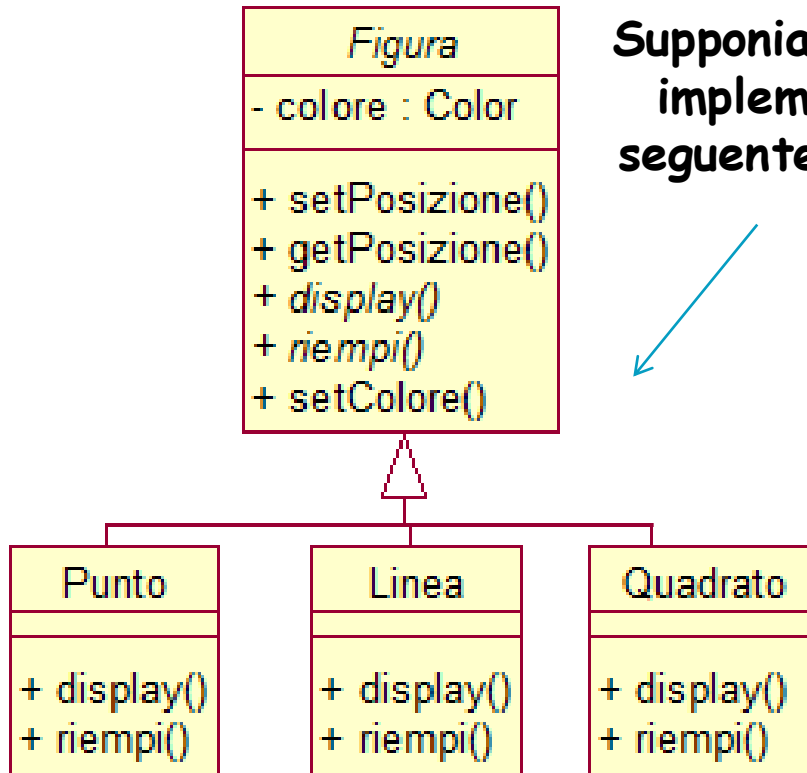
- usare una classe esistente, la cui interfaccia non è quella che il cliente si aspetta
- si vuole creare una classe che collabora con classi non correlate, o che non si conoscono ancora

ADAPTER – ESEMPIO (1)

Supponiamo di avere anche:



Supponiamo di avere implementato la seguente gerarchia:



◉ Vogliamo aggiungere **Cerchio**

- Lo implementiamo da zero?
 - ◉ Fatica inutile ...
- Usiamo la classe **Circle**?
 - ◉ Ha un'interfaccia diversa ...

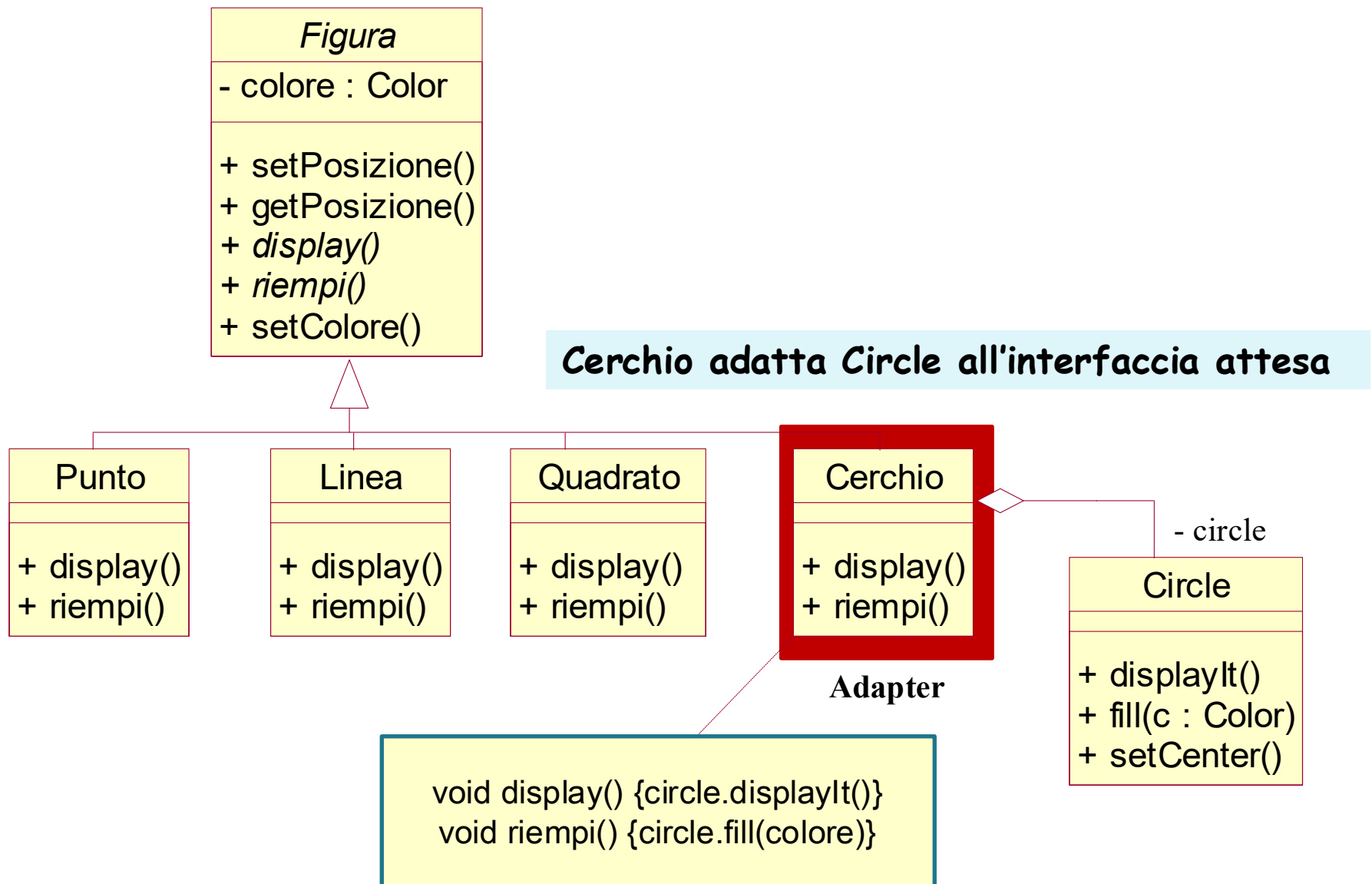
◉ e non possiamo modificare **Circle**

- non abbiamo il sorgente
- dobbiamo usarla così com'è (es. perchè usata da altre classi)



Soluzione: creare un adattatore e usare **Circle**!

ADAPTER – ESEMPIO (2)



DUE TIPI DI ADAPTER

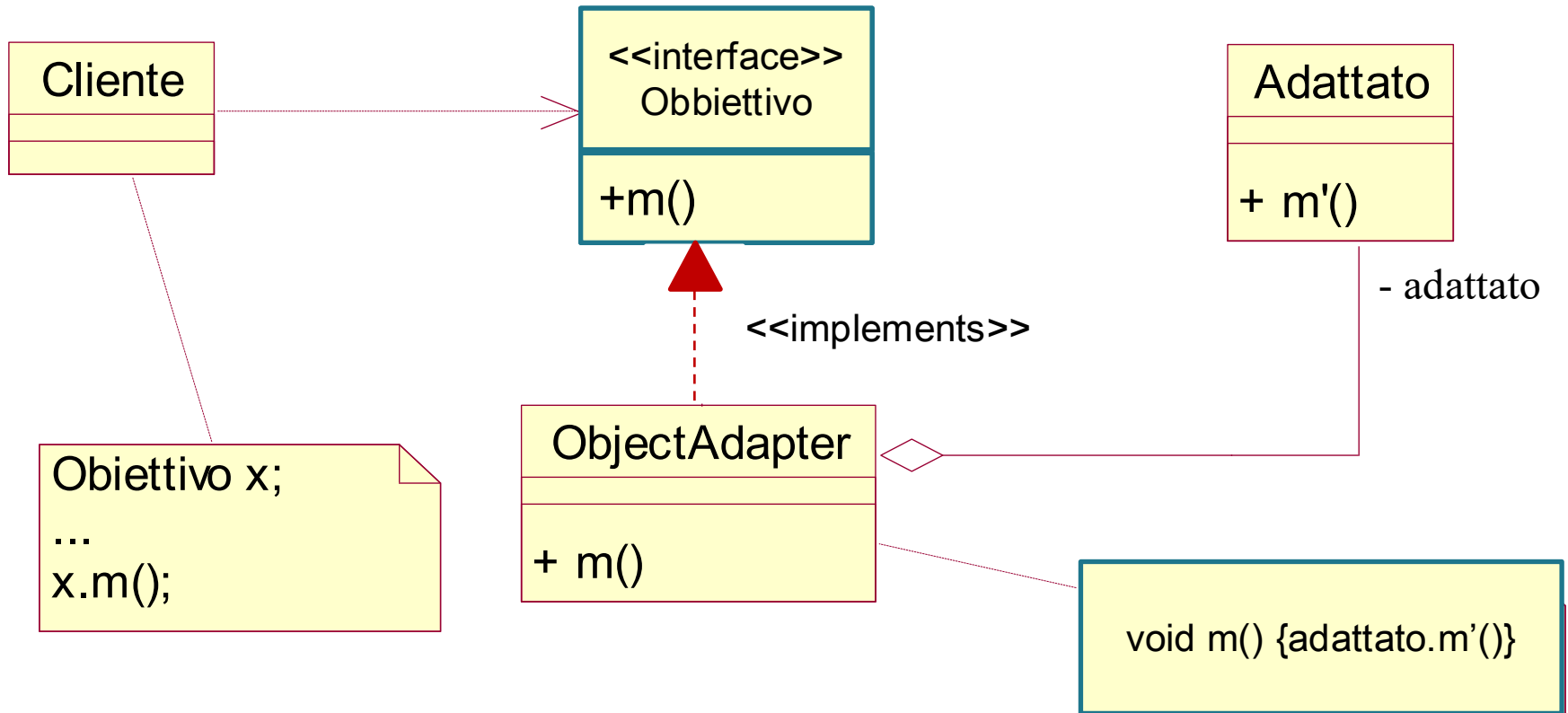
◦ Oggetto adattatore (Object Adapter)

- Basato su delega/composizione

◦ Classe adattatore (Class Adapter)

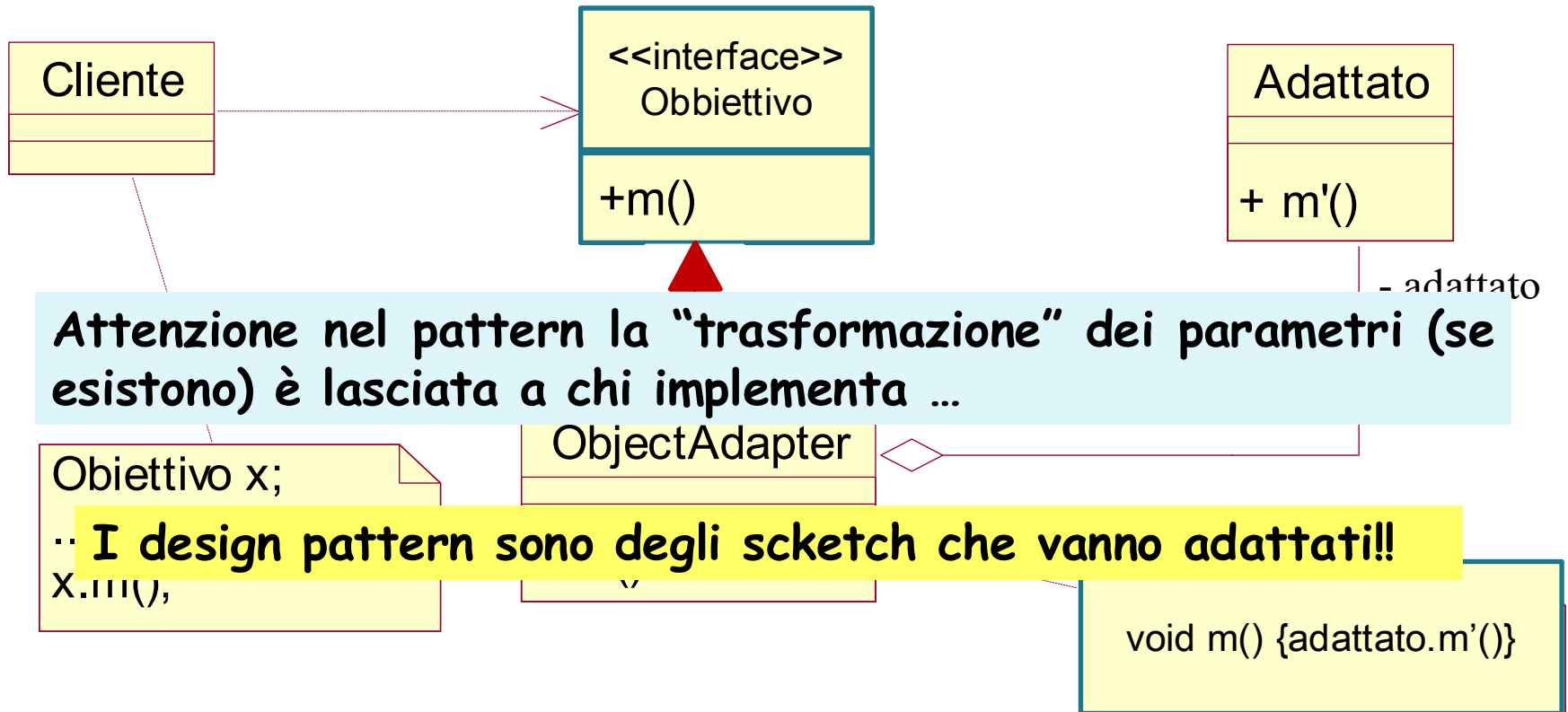
- Basato su ereditarietà
- L'adattatore eredita sia dall'interfaccia attesa sia dalla classe adattata
- No eredità multipla $\Rightarrow$ l'interfaccia attesa deve essere un'interfaccia, non una classe

DIAGRAMMA (OBJECT) ADAPTER



- ObjectAdapter converte, ovvero adatta, l'interfaccia che il cliente si aspetta (Obiettivo) all'interfaccia dell'adattato
- Il cliente usa ObjectAdapter come fosse l'oggetto Adattato
- ObjectAdapter possiede il riferimento a Adattato e sa come invocarlo

DIAGRAMMA (OBJECT) ADAPTER

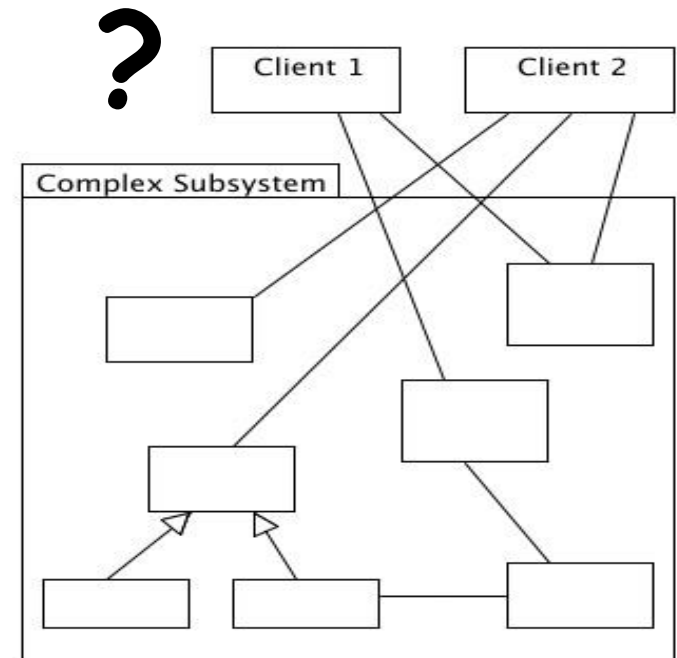


- ObjectAdapter converte, ovvero adatta, l'interfaccia che il cliente si aspetta (Obiettivo) all'interfaccia dell'adattato
- Il cliente usa ObjectAdapter come fosse l'oggetto Adattato
- ObjectAdatper possiede il riferimento a Adattato e sa come invocarlo

FAÇADE (FACCIATA)

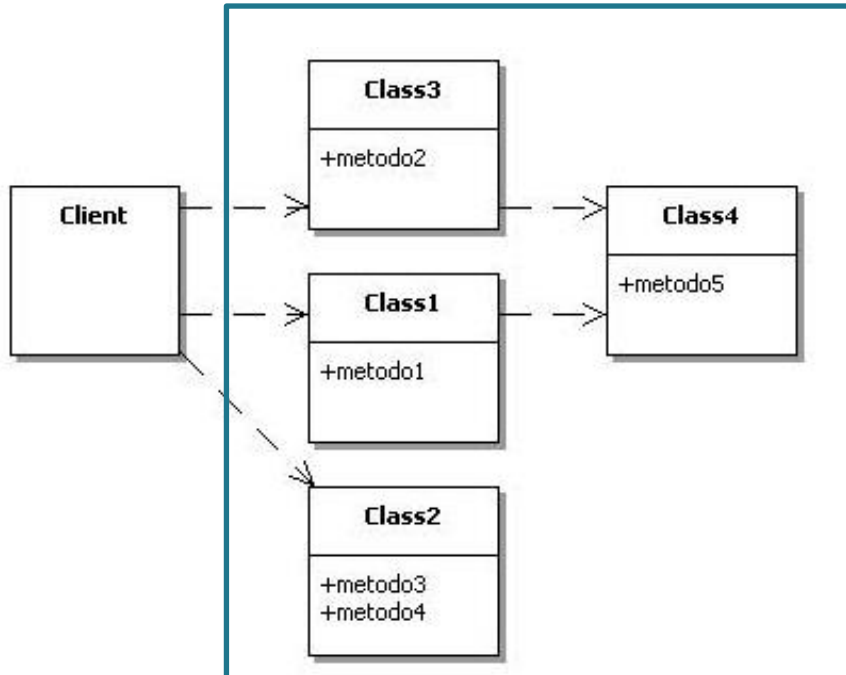
○ Problema

- Rendere più semplice l'accesso a sottosistemi che espongono interfacce complesse
- Fornire un'unica interfaccia per un insieme di funzionalità “sparse” su più interfacce/classi



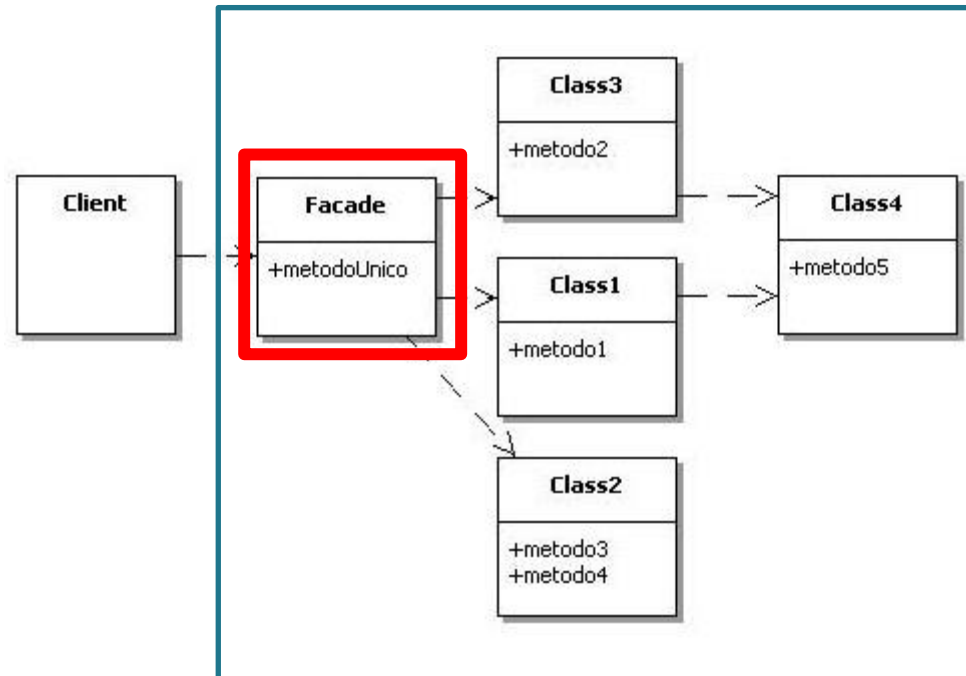
ESEMPIO 'CHIARIFICATORE'

Sottosistema



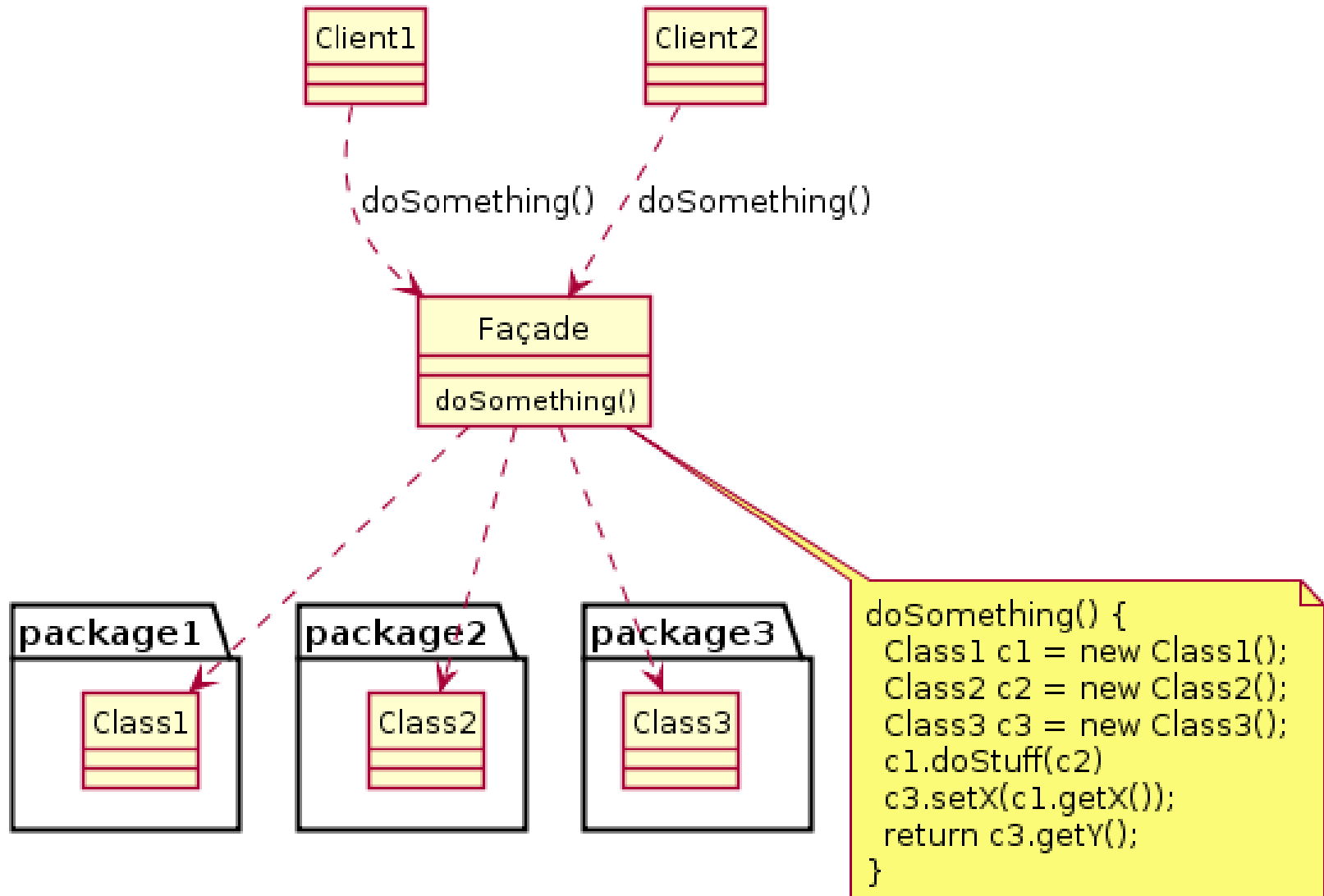
Un client per eseguire un task deve richiamare diversi metodi sparsi in classi diverse

Sottosistema



Facade nasconde la complessità poiché Client chiama solo *metodoUnico* per realizzare lo stesso task

ALTRO ESEMPIO



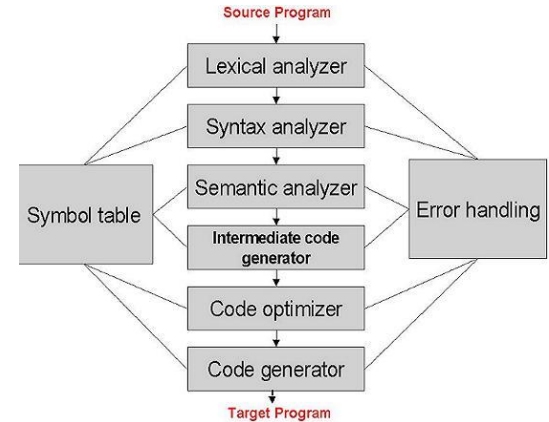
ESEMPI DI UTILIZZO

◉ Compilatore

- Classi: LexicalAnalyzer, SyntaxAnalyzer, SemanticAnalyzer, CodeGenerator, ecc.

vs.

- Classe **Compiler** con metodo compile()

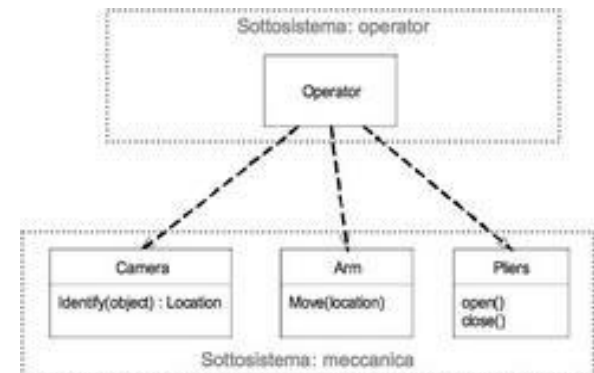


◉ Robot

- Classi: Camera (per identificare gli oggetti), Arm (per muovere) e Pliers (per afferrare)

vs.

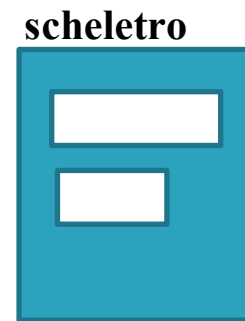
- Classe **Robot** con metodo moveObject()



CONSEGUENZE: FAÇADE PATTERN

- Promuove un accoppiamento debole fra cliente e sottosistema 😊
 - Riduce la dipendenza tra client e sottosistema
- Nasconde al cliente le componenti (complesse) del sottosistema 😊
- Il cliente può comunque, se necessario, usare direttamente le classi del sottosistema 😊
- **Façade vs. Adapter**
 - Entrambi sono “wrapper” (involucri)
 - Entrambi si basano su un'interfaccia, ma:
 - **Façade la semplifica**
 - **Adapter la converte**

TEMPLATE METHOD



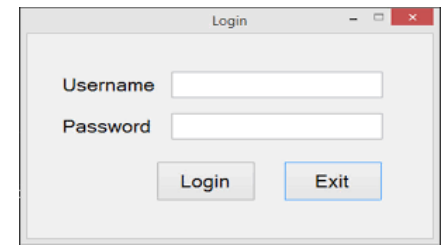
Passi definiti nelle
sottoclassi

- Definisce lo **scheletro (template)** di un **algoritmo in un metodo** posponendo la definizione di alcuni passi a delle sottoclassi
 - permette di ridefinire alcuni passi di un algoritmo senza cambiare lo scheletro

○ Problema

- Implementare la parte **invariante** di un algoritmo una sola volta, e lasciare alle sottoclassi l'implementazione delle parti che possono variare
 - per evitare duplicazioni

TEMPLATE METHOD - ESEMPIO



- Vogliamo scrivere una classe **riusabile** per il **logging degli utenti** in una applicazione, che svolga i seguenti passi:

1. Chieda all'utente id e pwd
2. Autentichi l'utente producendo un oggetto che incapsuli eventuali informazioni che potranno essere richieste in seguito come prova dell'autenticazione
3. Mostri all'utente un display animato mentre l'autenticazione è in corso
4. Notifichi all'utente quando il login è completo, rendendo l'oggetto prodotto dall'autenticazione disponibile all'app

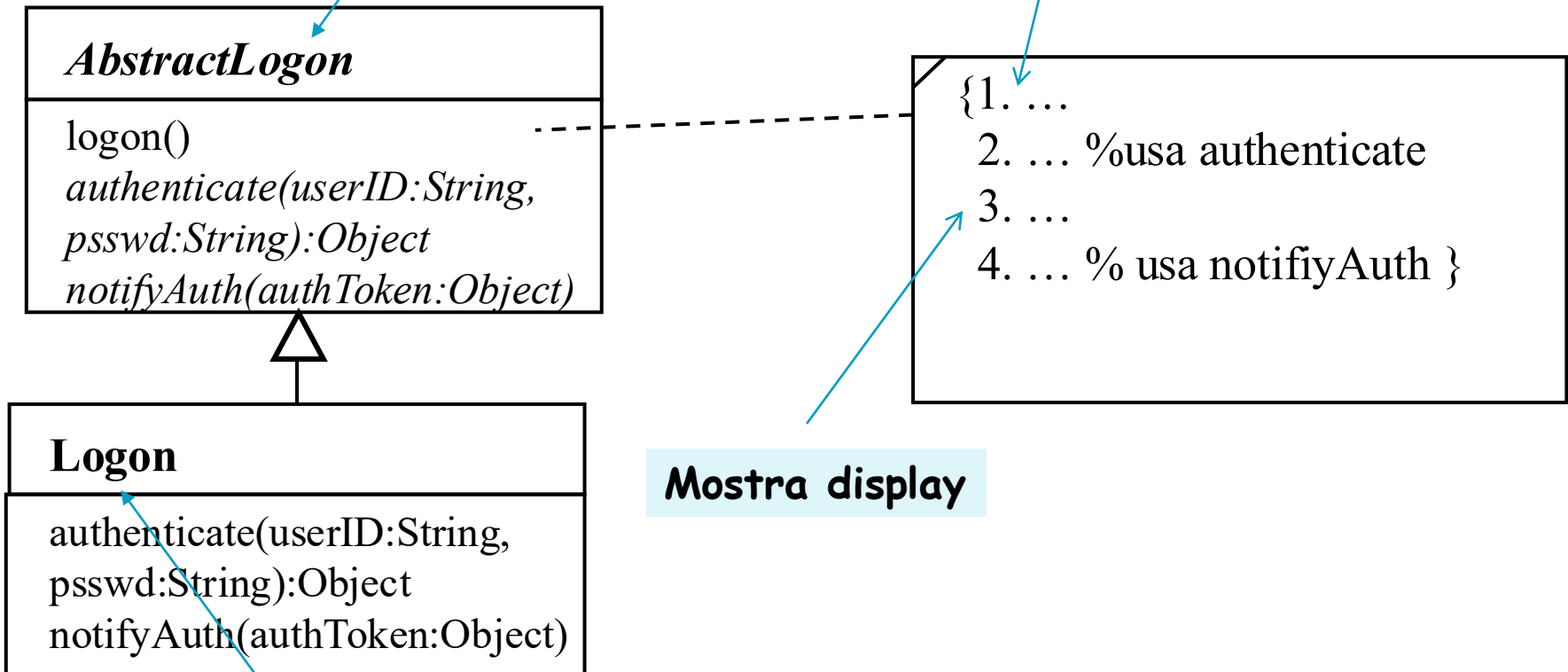
Parte invariante/fissa (anche se possono cambiare messaggi e immagini, la logica è la stessa)

Parte variabile (la logica dipende dall'applicazione)

TEMPLATE METHOD

Classe astratta (riusabile)

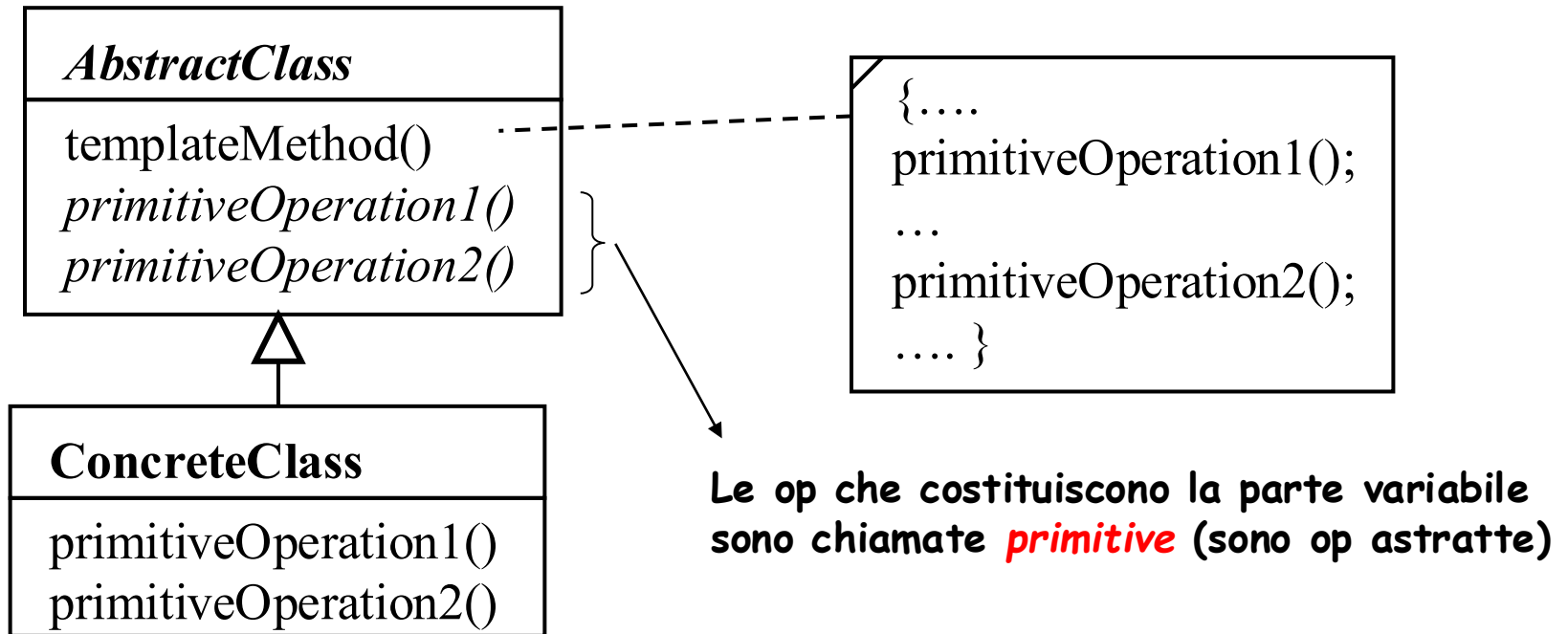
Chiede id e pwd



Mostra display

Classe concreta che fa override di authenticate() e notifyAuth()

IL PATTERN REALIZZA INVERSIONE DI CONTROLLO



Hollywood principle (inversione di controllo)

Non chiamate, richiameremo noi ...

Normalmente sono le sottoclassi a chiamare i metodi delle superclassi. Con questo pattern è `templateMethod()` a chiamare i metodi specifici ridefiniti nelle sottoclassi

CONSEGUENZE: TEMPLATE METHOD

- Tecnica fondamentale per il riuso del codice
 - molto usata nelle librerie e nei **framework**

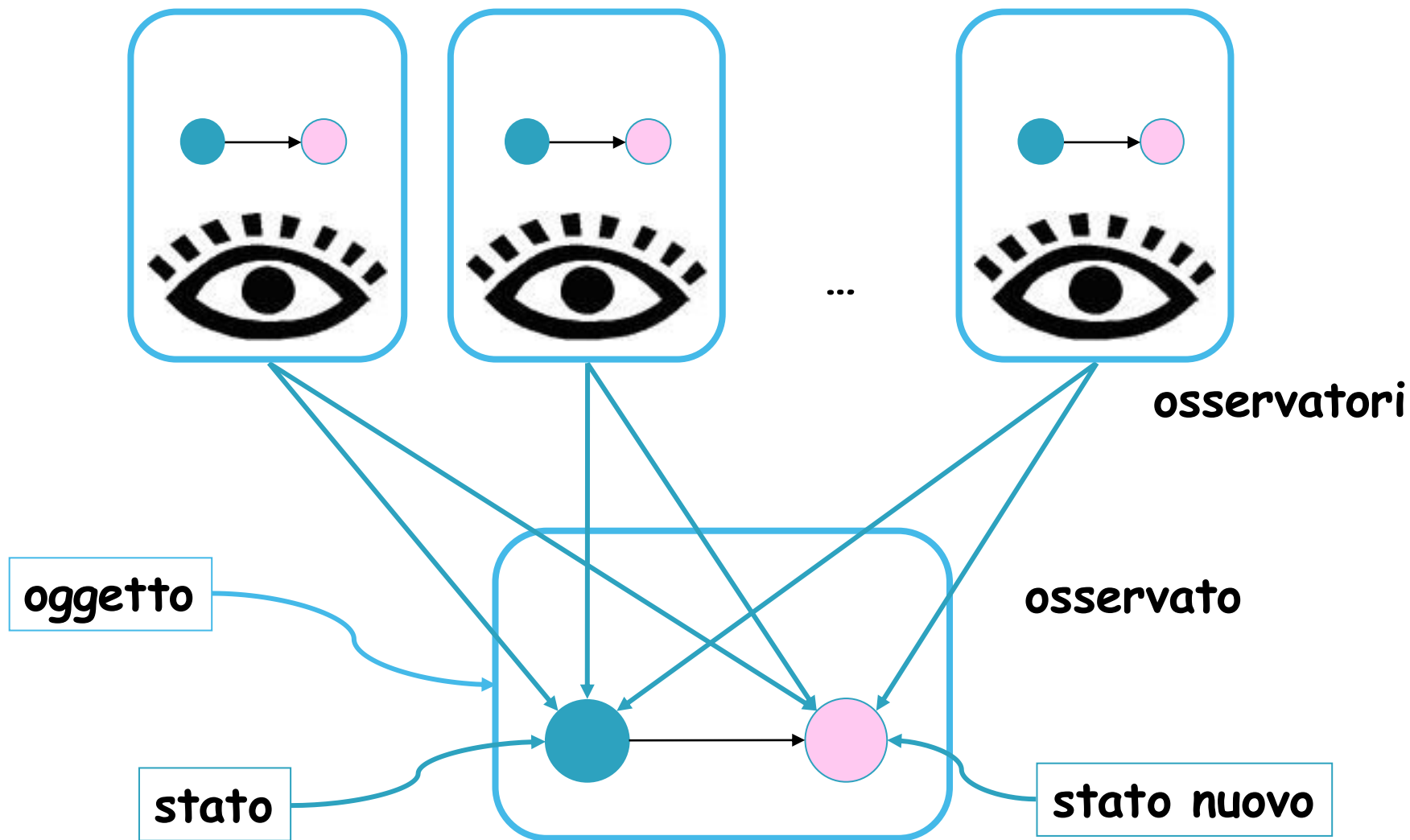


Reuse reduces the risks and costs of software development and maintenance. Figure 2

- Realizza inversione del flusso di controllo
- Permette di avere anche più sottoclassi concrete
 - Rappresentano differenti parti variabili
 - Ad esempio Logon1, Logon2, Logon3
- Importante chiarire bene quali operazioni devono essere ridefinite nelle sottoclassi
 - Parte fissa vs. parte variabile



OBSERVER - COSA VORREMMO



PERÒ COME LO IMPLEMENTIAMO?

- Attributi di stato pubblici?!?
 - Non scherziamo ... 
- **Con un operazione:** gli osservatori invocano “continuamente” un operazione **cambiato()** dell'oggetto osservato e poi **getStato()**

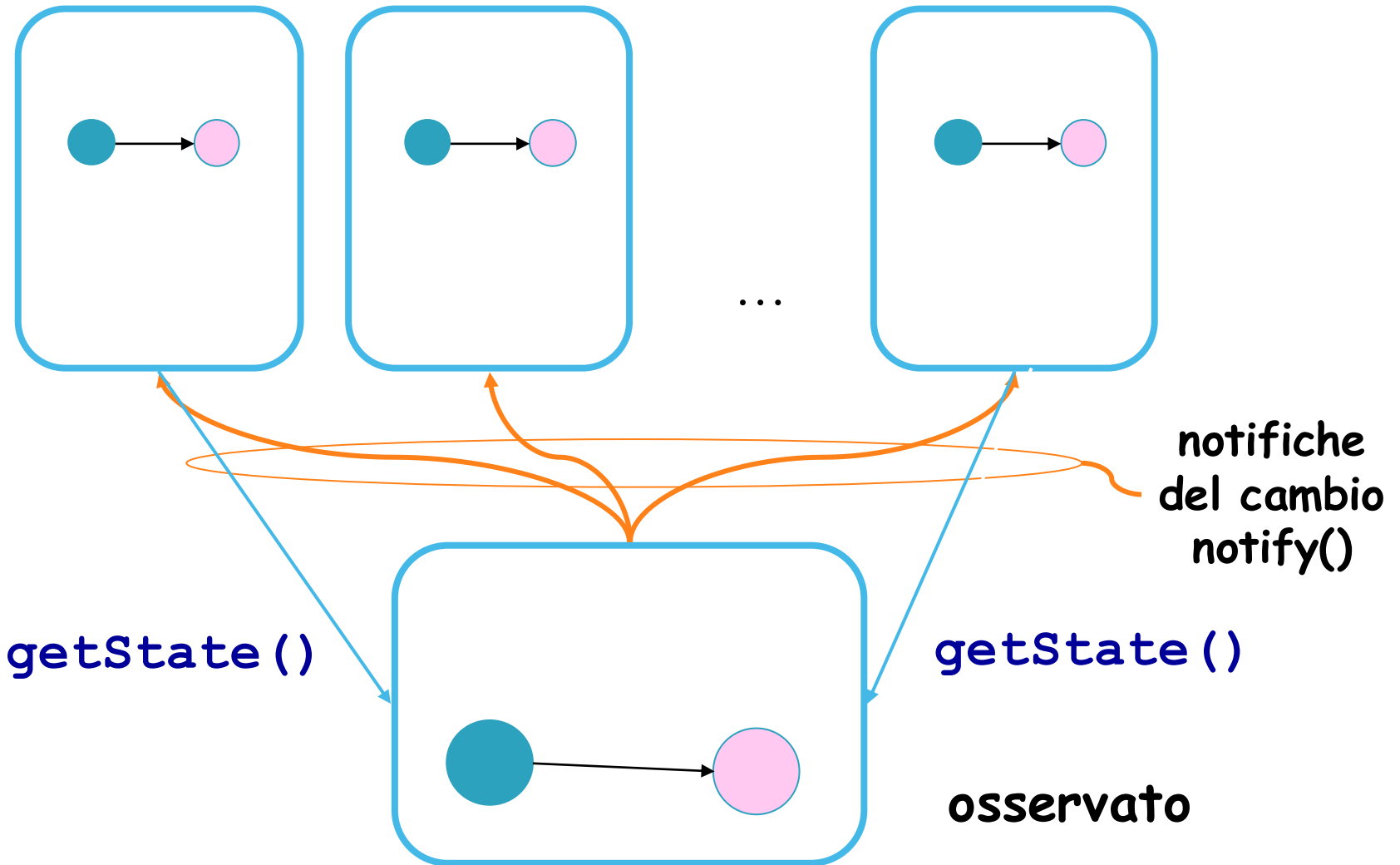
OggettoOsservato
stato: Stato
cambiato(): boolean

?

- Così l'osservatore potrebbe:
 - interrogare troppe volte l'osservato
 - scoprire la variazione troppo tardi
 - perdersi la prima di due variazioni vicine
- E in più se ho tanti osservatori? **poco scalabile**
 - l'osservato passa il tempo a rispondere (magari senza nessuna variazione)

L'IDEA ...

osservatori

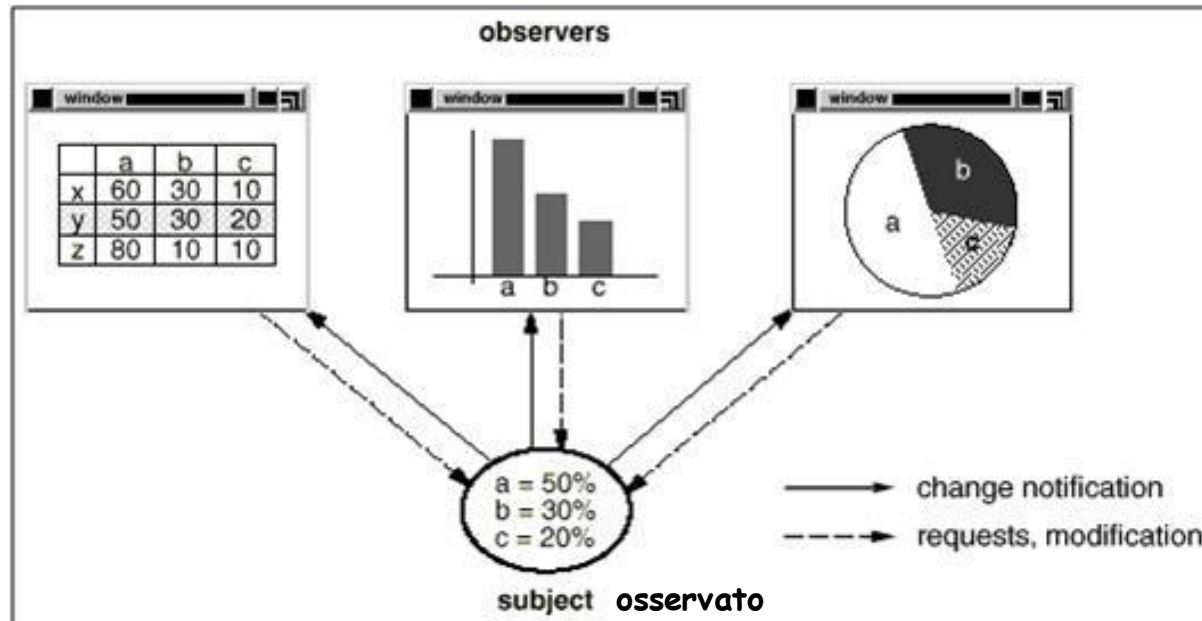


QUINDI ...

- Gli osservatori si **registrano** presso l'oggetto osservato
 - Gli osservatori si possono aggiungere e togliere dinamicamente (**runtime**)
- Quando l'oggetto osservato cambia stato, **notifica** tutti gli osservatori
 - invoca l'operazione **notify()**
- Quando notificato, ogni osservatore decide cosa fare:
 - Niente
 - **Richiedere** all'osservato informazioni sullo stato
 - **getState()**
 - ...

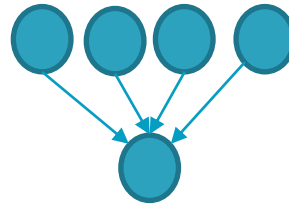
VISTE SULLO STESSO OGGETTO

- Gli osservatori potrebbero essere varie “viste” sullo stesso oggetto
 - Dati di un foglio elettronico e loro rappresentazioni tabellari o grafiche



Modello e viste ad esso collegate sono consistenti!

OBSERVER



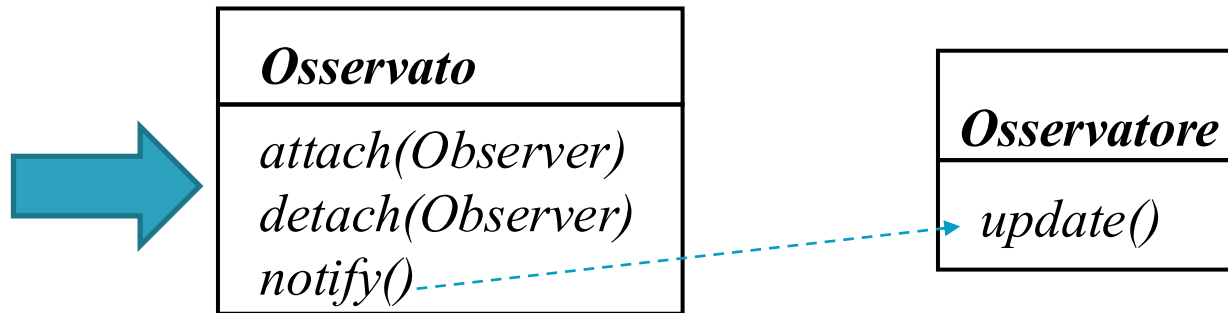
- Definisce una **dipendenza “lasca”** uno a molti tra oggetti, in modo che quando un oggetto cambia stato, tutti gli oggetti che dipendono da lui sono avvertiti ...

Problema

- Associare più “viste” differenti ad un modello (dati)
- Implementare il **broadcast**
- Il cambiamento di un oggetto richiede il cambiamento di altri oggetti
 - Non si conosce quanti oggetti devono cambiare
- Notificare oggetti senza fare assunzioni a priori su quali siano questi oggetti
 - Evita l'accoppiamento “forte”

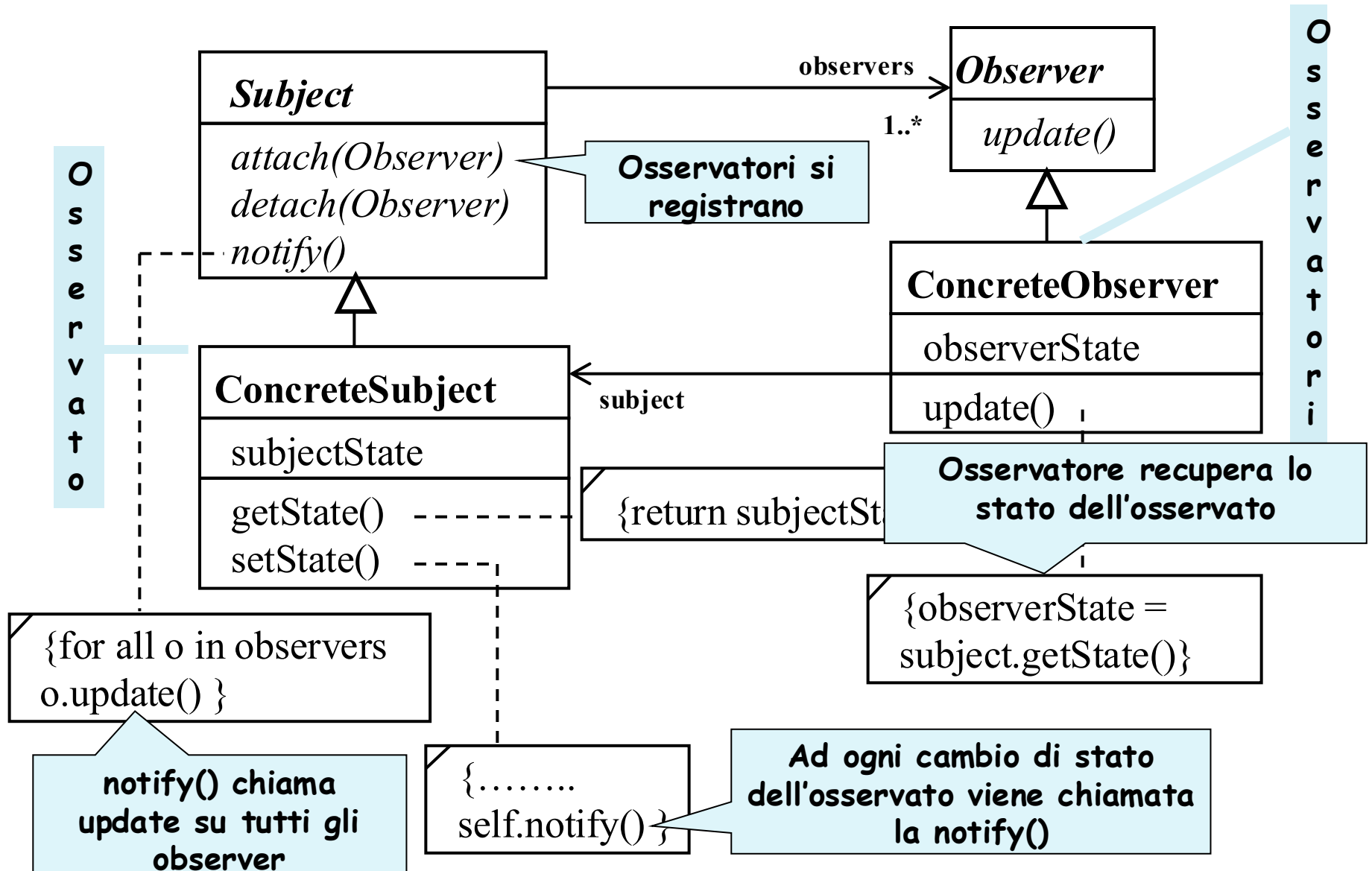
INTERFACCE

- L'oggetto osservato deve fornire un'interfaccia standard per la registrazione `attach(Osservatore)` e `detach(Osservatore)`

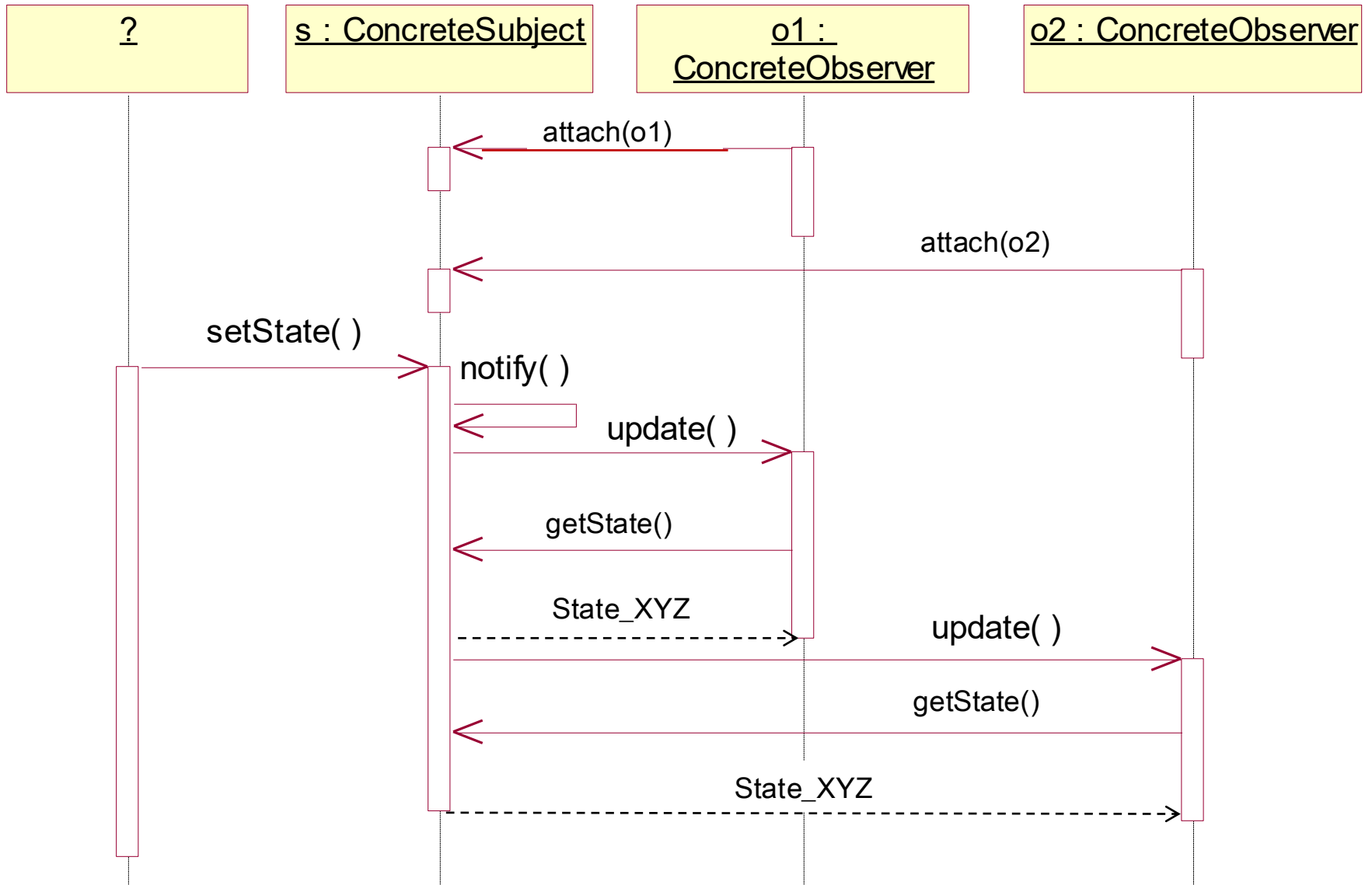


- L'oggetto osservato deve poter notificare:
 - l'oggetto osservato ha `notify()` che chiama tutti gli `update()` dei registrati
 - gli osservatori devono fornire un'interfaccia standard per la notifica `update()`

OBSERVER DESIGN PATTERN



SEQUENCE DIAGRAM OBSERVER



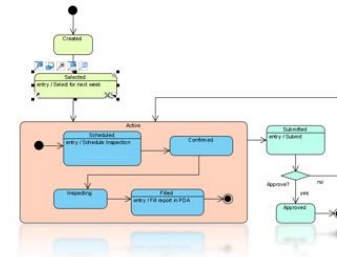
CONSEGUENZE: OBSERVER PATTERN

- Collegamento lasco tra Osservato e Osservatori 😊
 - L'Osservato sa che vi è una lista di Osservatori, ma non conosce la loro struttura (è **dinamica**)
 - Osservato e Osservatori possono appartenere a diversi strati di astrazione in un sistema
- Supporto per la **comunicazione broadcast** 😊
 - publish/subscribe
- Modifiche inaspettate 😞
 - una modifica innocente nell'osservato può scatenare una cascata di modifiche negli Osservatori e nei loro dipendenti

PushButton(eroga caffè) → msg (stato=soldi inseriti insuff)

Scopo

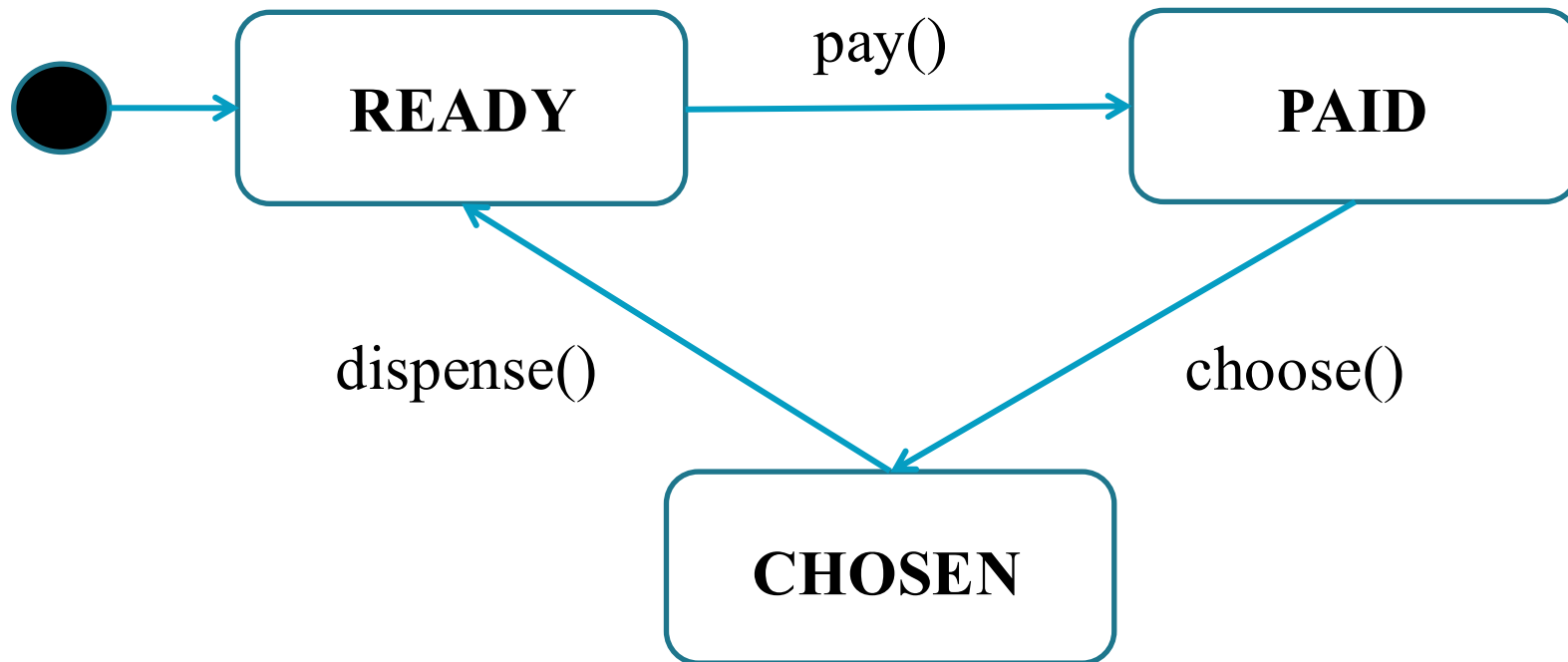
- Permettere a un oggetto di cambiare il suo comportamento al variare del suo stato interno
- **Permette di implementare le state machine** (bene!) in un linguaggio che non le supporta nativamente



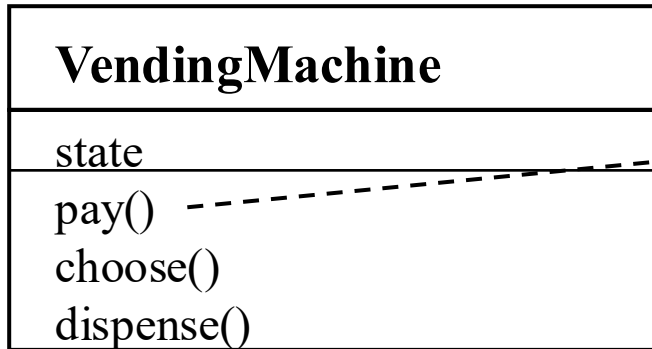
○ Come funziona?

- Si estrae la **rappresentazione dello stato** in classi esterne
 - organizzate in una gerarchia!
- Si sfrutta il **polimorfismo** per variare il comportamento

ESEMPIO: VENDING MACHINE



POSSIBILE IMPLEMENTAZIONE (IF/SWITCH)



```
{...  
if (state==READY )  
...  
else if (state == PAID)  
...  
else if (state == CHOSEN)}
```

ESEMPIO:

```
public void pay(money) {  
    switch(state) {  
        case READY:  
            if (money are sufficient) {  
                ... cambia il totale ... msg("money ok") ...  
                state=PAID;  
            } else { // money not sufficient  
                ... cambia il totale ... msg("insert more money") ...  
            }  
        PAID  
        CHOSEN  
        default: msg("Op not possibile!!");  
    }
```


PERÒ ...

Tanti if... (campanello d'allarme)



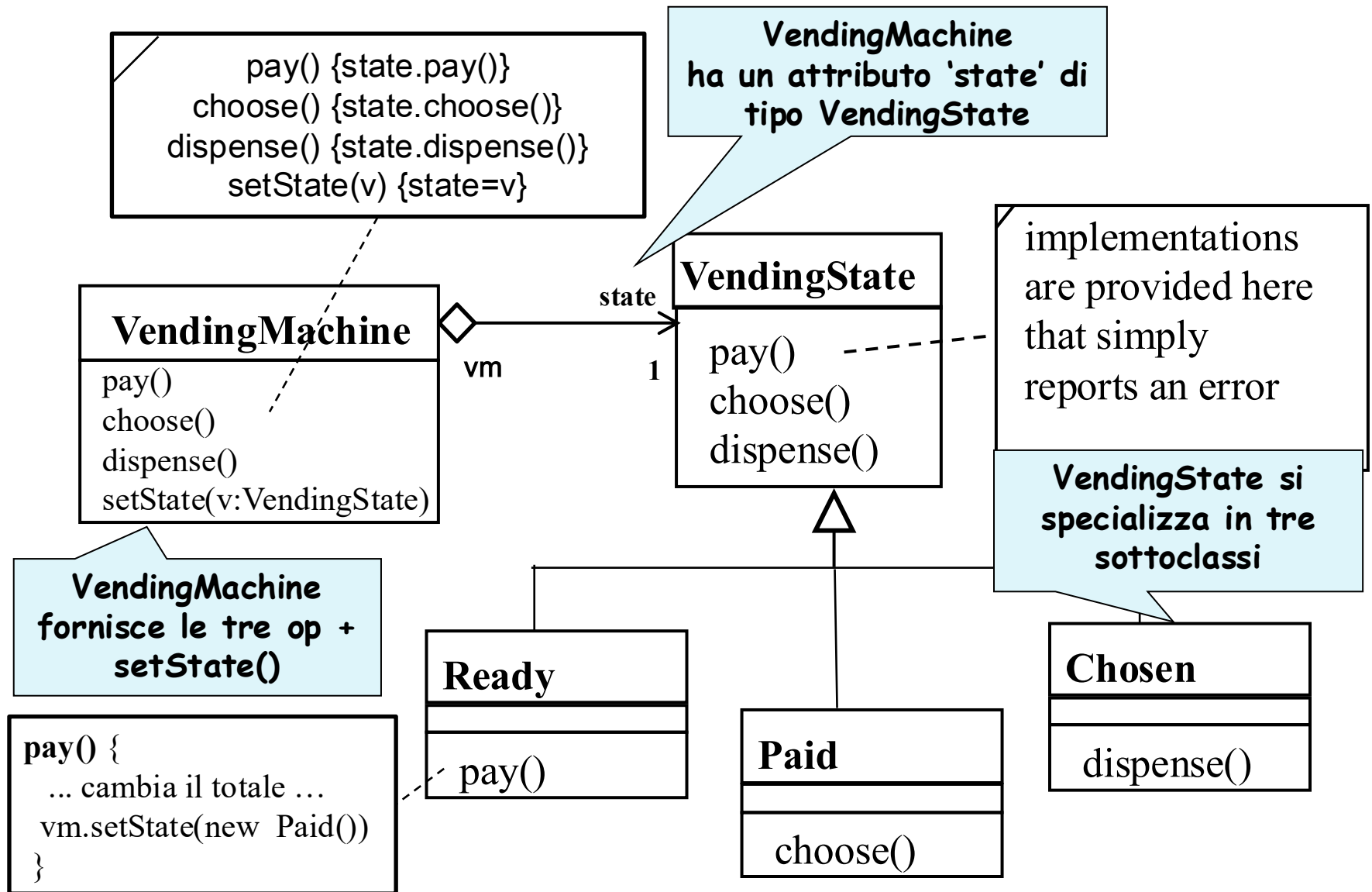
- Difficile da comprendere e mantenere

- Aggiunta di uno stato implica molte modifiche
 - Un 'If' per ogni metodo di evento

- **Alternativa: State pattern!**

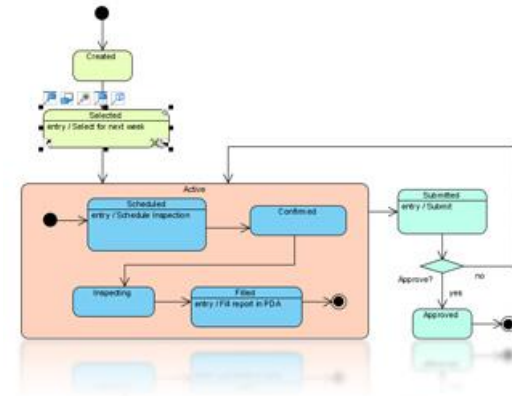
- Usiamo una gerarchia di classi per rappresentare gli stati della macchina
- Ogni (sotto)classe uno stato
- Ogni (sotto)classe ha la sua implementazione delle operazioni

IMPLEMENTAZIONE CON STATE PATTERN



Sovrascriviamo solo le operazioni "che hanno un significato" in quello stato

STATE PATTERN



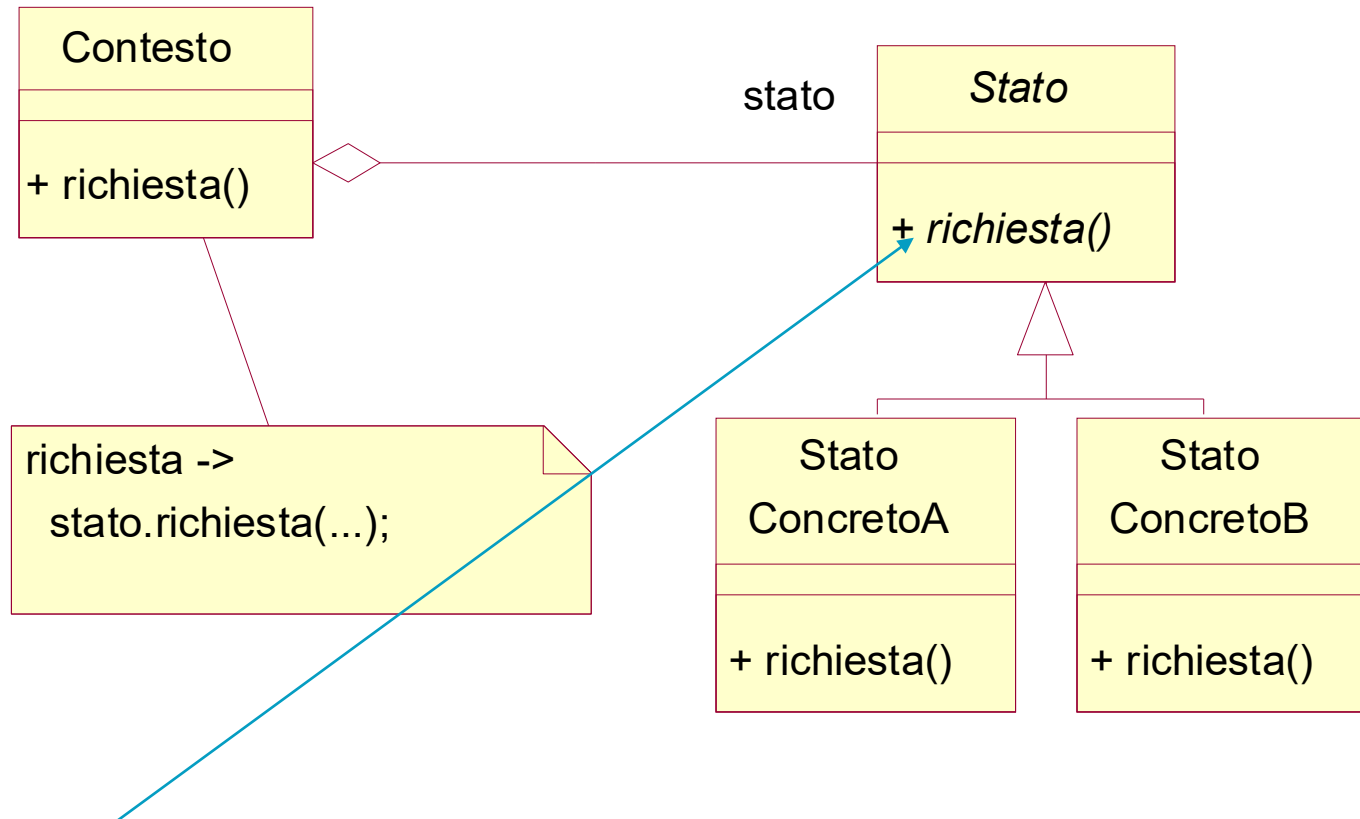
Consente ad un oggetto di cambiare il proprio comportamento a run-time in funzione dello stato in cui si trova

- Es. Vending Machine

◦ Problema

- Un oggetto deve modificare dinamicamente il suo comportamento al variare del suo stato interno
 - State machine

DIAGRAMMA STATE PATTERN



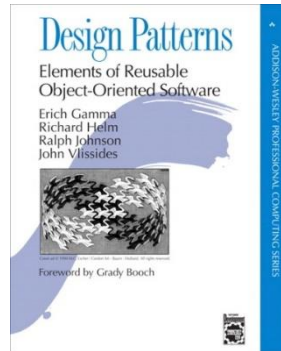
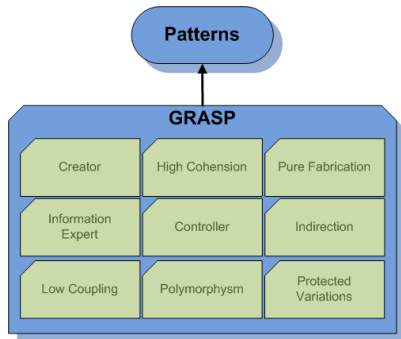
Se richiesta è astratto nella superclasse (astratta) Stato, le sottoclassi concrete devono implementare anche i metodi che corrispondono a transizioni non ammesse in quello stato (che solleveranno errori)

CONSEGUENZE: STATE PATTERN

- Incapsula i comportamenti associati ad uno stato in un oggetto e la logica di transizione tra stati in uno oggetto piuttosto che in una grosso switch tra istruzioni
 - La soluzione “switch” è difficile da mantenere/estendere
- Il comportamento associato ad uno stato dipende solo da una classe
 - *StatoConcretoA, vedi diagramma*
- E' semplice cambiare il comportamento di uno specifico stato e aggiungere nuovi stati
- Incrementa il numero delle classi



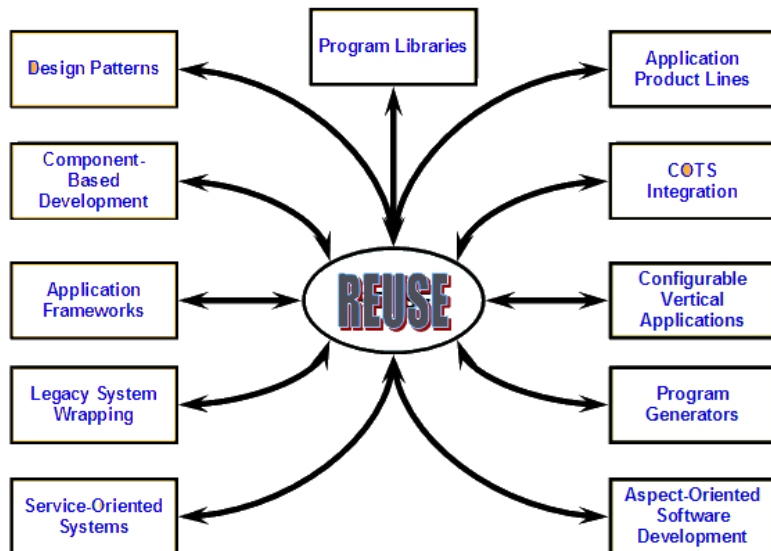
RIASSUMENDO



DESCRIZIONE DI UN PATTERN

- Template piuttosto preciso!
 - Versione semplificata (solo parti fondamentali)

- **Nome (una/due parole)**
 - Significativo/esplicativo, che aiuta a capire a cosa serve il pattern
- **Problema**
 - descrive **quando** applicare il pattern
 - può contenere anche delle precondizioni per poterlo applicare
- **Soluzione** **Non sempre precisa e completa, è uno sketch**
 - descrive **astrattamente** gli elementi del design proposto, le loro relazioni, responsabilità e collaborazioni
- **Conseguenze**
 - Pro/contro della sua applicazione



Creazione	Struttura	Comportamento
Factory Method Abstract Factory Builder <u>Prototype</u> Singleton	Adapter Bridge Composite Decorator <u>Facade</u> <u>Flyweight</u> Proxy	Interpreter Template Method Chain of responsibility <u>Command</u> <u>Iterator</u> Mediator Memento Observer State Strategy Visitor